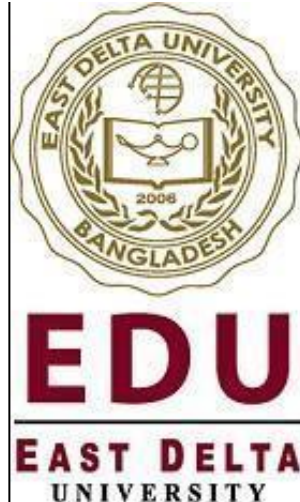




Multi-Stage Vehicle Detection Under Foggy Conditions: A Comparative Study of YOLOv8s and YOLOv10s

By

Md. Shahidul Islam Sakib
223001712

Department of Computer Science & Engineering
School of Science, Engineering & Technology

Supervised by

Tahmina Akter Sumi

Lecturer

Department of Computer Science & Engineering

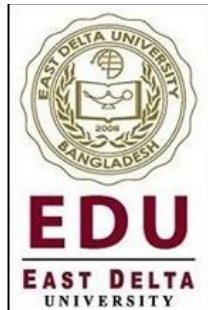
In partial fulfillment of the requirement for the degree of *Bachelor of
Science in Computer Science and Engineering*

September, 2026

East Delta University
Noman Society, East Nasirabad, Chittagong-4209

Multi-Stage Vehicle Detection Under Foggy Conditions: A Comparative Study of YOLOv8s and YOLOv10s

This thesis is submitted in partial fulfillment of the requirement for the degree of
Bachelor of Science in Computer Science & Engineering.



By

Md. Shahidul Islam Sakib
223001712

Supervised by

Tahmina Akter Sumi

Lecturer

Department of Computer Science & Engineering

Department of Computer Science & Engineering

School of Science, Engineering & Technology

East Delta University, Noman Society, East Nasirabad, Chittagong-4209

The undergraduate thesis titled **Multi-Stage Vehicle Detection Under Foggy Conditions: A Comparative Study of YOLOv8s and YOLOv10s** submitted by Md. Shahidul Islam Sakib, Id: 223001712 has been accepted as satisfactory in fulfillment of the requirement for the degree of Bachelor of Science (B.Sc.) in Computer Science and Engineering (CSE) to be awarded by East Delta University.

Board of Examiners

Saraf Anika

Assistant Professor

School of Science, Engineering and Technology

East Delta University

Chairman

Md. Jamil Uddin

Lecturer

School of Science, Engineering and Technology

East Delta University

Reviewer

Tahmina Akter Sumi

Lecturer

School of Science, Engineering and Technology

East Delta University

Supervisor

Declaration

It is hereby declared that the work contained in this thesis is original. The information derived from the literature or work has been duly acknowledged and presented in the reference section. No part of this thesis has been submitted elsewhere for the degree, diploma or other similar title of recognition.

Date:[Enter Date]

Md Shahidul Islam Sakib
Student Id: 223001712

ACKNOWLEDGMENTS

All praise belongs to Almighty Allah for granting me the strength, patience, and opportunity to complete this thesis.

I would like to express my deepest gratitude to my supervisor, Tahmina Akter Sumi, for their invaluable guidance, continuous support, and insightful feedback throughout this research. Their expertise and encouragement were instrumental in shaping this work on vehicle detection under adverse foggy conditions.

I am also grateful to the Department of Computer Science and Engineering, East Delta University, for providing the academic environment and resources necessary to carry out this research.

My sincere thanks go to my family, and especially my wife, for their unwavering support, patience, and encouragement throughout my academic journey, and for standing by me during the demanding final months of this thesis

I would also like to acknowledge the open-source community and the developers of the Cityscapes and Foggy Cityscapes datasets, as well as the Ultralytics YOLO framework, whose tools and resources made this research possible.

Finally, I extend my appreciation to my friends and colleagues who offered their encouragement and constructive discussions during this work.

AI Assistance Disclosure

I used AI tools during the preparation of this thesis for the following purposes:

- **Research/Idea Generation:** DeepSeek and Perplexity were used to assist with literature search and background research.
- **Coding/Technical Work:** Claude was used to assist with supporting diagrams and verification of technical/architectural details.
- **Writing/Editing:** Claude was used to assist with drafting, structuring, and editing the written content of this thesis.
- **Other Assistance:** N/A

AI Tool(s) Used: DeepSeek, Perplexity, Claude (Anthropic)

I confirm that all AI-generated or AI-assisted outputs used in this thesis were reviewed and critically evaluated by me. Where applicable, AI-assisted code was tested and verified before use. I remain responsible for the content and presentation of this thesis.

TABLE OF CONTENTS

Declaration	iv
ACKNOWLEDGMENTS	v
AI Assistance Disclosure	vi
TABLE OF CONTENTS	vii
LIST OF TABLES	x
LIST OF FIGURES	xi
LIST OF ABBREVIATIONS	xiii
ABSTRACT	xv
1 CHAPTER 1 INTRODUCTION	1
1.1 Background of the Study	1
1.2 Problem Statement	2
1.3 Research Motivation	2
1.4 Research Objectives	4
1.4.1 General Objective	4
1.4.2 Specific Objectives	4
1.5 Research Questions	4
1.6 Scope of the Study	5
1.7 Significance of the Study	5
1.8 Thesis Organization	6
2 CHAPTER 2 LITERATURE REVIEW	7
2.1 Introduction	7
2.2 Intelligent Transportation Systems	7
2.3 Vehicle Detection	7

2.4	Object Detection in Computer Vision	8
2.5	Deep Learning for Object Detection	9
2.6	YOLO Architecture	9
2.6.1	Evolution of YOLO	9
2.6.2	YOLOv8	10
2.6.3	YOLOv10	10
2.7	Challenges of Vehicle Detection in Adverse Weather	10
2.7.1	Fog	10
2.7.2	Rain	11
2.7.3	Snow	11
2.7.4	Low-Light Conditions	12
2.8	Fog Image Enhancement Techniques	12
2.9	Related Works	13
2.9.1	Traditional Approaches	13
2.9.2	Deep Learning-Based Approaches	13
2.9.3	Recent Studies (2023–2025)	15
2.10	Research Gap	16
2.11	Chapter Summary	17
3	CHAPTER 3 METHODOLOGY	18
3.1	Dataset	18
3.1.1	Class Distribution	19
3.2	Preprocessing	20
3.2.1	Annotation Cleaning	20
3.2.2	Gamma Correction and CLAHE	20
3.2.3	Rare-Class Oversampling	21
3.3	Curriculum Training Pipeline	22
3.3.1	YOLOv8s Training Configuration	22
3.3.2	YOLOv10s Training Configuration	22
3.4	Models	24
3.4.1	YOLOv8s Architecture	25
3.4.2	YOLOv10s Architecture	28

3.5	Hardware/Environment	33
3.6	Evaluation	34
3.6.1	Evaluation Metrics	34
3.6.2	Evaluation Protocol	34
3.6.3	Best-Epoch Selection	34
4	CHAPTER 4 RESULTS AND DISCUSSION	35
4.1	Results	35
4.1.1	YOLOv8s Results	35
4.1.2	YOLOv10s Results	41
4.1.3	Qualitative Detection Examples	47
4.1.4	Training Convergence Curves	53
4.2	Comparative Analysis: YOLOv8s vs. YOLOv10s	58
4.2.1	Quantitative Comparison: Per-Class Detail	59
4.2.2	Computational Efficiency Comparison	60
4.2.3	Qualitative Comparison	61
4.3	Comparison with Published Benchmarks	62
4.4	Discussion	64
4.2	Practical Implications	65
5	CHAPTER 5 CONCLUSION AND FUTURE WORK	67
5.1	Summary of the Study	67
5.2	Achievement of Research Objectives	67
5.3	Major Findings	68
5.4	Limitations	68
5.5	Recommendations	69
5.6	Future Work	70
5.7	Final Remarks	71
	References	72
	Appendix	77

LIST OF TABLES

2.1	Key papers discussed in this section, by approach category	15
3.1	Class-wise image and instance count on the validation split	20
3.2	YOLOv8s training hyperparameters	24
3.3	YOLOv10s training hyperparameters	24
3.4	YOLOv8s backbone layer sequence (“s” scale)	25
3.5	YOLOv10s backbone layer sequence and module types	29
4.1	YOLOv8s overall performance by stage	35
4.2	YOLOv8s per-class mAP50 by stage	36
4.3	YOLOv10s overall performance by stage	41
4.4	YOLOv10s per-class mAP50 by stage	42
4.5	Overall mAP50 comparison between YOLOv8s and YOLOv10s by stage	58
4.6	Per-class mAP50 comparison at Clean and $\beta = 0.02$, both models	59
4.7	mAP50 (%) at heavy fog against HR-YOLO’s proposed model	63

LIST OF FIGURES

3.1	Sample scene at the four visibility conditions	19
3.2	Effect of gamma correction and CLAHE applied to a foggy training image	21
3.3	Detailed curriculum training pipeline for YOLOv8s and YOLOv10s	23
3.4	YOLOv8s network architecture: backbone, neck, and multi-scale detection head	28
3.5	Illustration of the consistent dual assignment strategy	32
3.6	YOLOv10s network architecture	33
4.1	Normalized confusion matrix for YOLOv8s, clean condition	37
4.2	Normalized confusion matrix for YOLOv8s, $\beta = 0.005$ condition	37
4.3	Normalized confusion matrix for YOLOv8s, $\beta = 0.01$ condition	38
4.4	Normalized confusion matrix for YOLOv8s, $\beta = 0.02$ condition	38
4.5	Precision-recall curve for YOLOv8s, Clean condition	39
4.6	Precision-recall curve for YOLOv8s, $\beta = 0.005$ condition	39
4.7	Precision-recall curve for YOLOv8s, $\beta = 0.01$ condition	40
4.8	Precision-recall curve for YOLOv8s, $\beta = 0.02$ condition	40
4.9	Normalized confusion matrix for YOLOv10s, Clean condition	43
4.10	Normalized confusion matrix for YOLOv10s, $\beta = 0.005$ condition	43
4.11	Normalized confusion matrix for YOLOv10s, $\beta = 0.01$ condition	44
4.12	Normalized confusion matrix for YOLOv10s, $\beta = 0.02$ condition	44
4.13	Precision-recall curve for YOLOv10s, Clean condition	45
4.14	Precision-recall curve for YOLOv10s, $\beta = 0.005$ condition	45
4.15	Precision-recall curve for YOLOv10s, $\beta = 0.01$ condition	46
4.16	Precision-recall curve for YOLOv10s, $\beta = 0.02$ condition	46
4.17	Ground-truth bounding boxes for a representative Clean-condition validation batch	48
4.18	Ground-truth bounding boxes across the three foggy conditions	48
4.19	YOLOv8s predicted detections, clean condition	49

4.20	YOLOv8s predicted detections, $\beta = 0.005$ condition	49
4.21	YOLOv8s predicted detections, $\beta = 0.01$ condition	50
4.22	YOLOv8s predicted detections, $\beta = 0.02$ condition	50
4.23	YOLOv10s predicted detections, clean condition	51
4.24	YOLOv10s predicted detections, $\beta = 0.005$ condition	51
4.25	YOLOv10s predicted detections, $\beta = 0.01$ condition	52
4.26	YOLOv10s predicted detections, $\beta = 0.02$ condition	52
4.27	Training curves for YOLOv8s, Clean stage	53
4.28	Training curves for YOLOv8s, $\beta = 0.005$ stage	54
4.29	Training curves for YOLOv8s, $\beta = 0.01$ stage	54
4.30	Training curves for YOLOv8s, $\beta = 0.02$ stage	55
4.31	Training curves for YOLOv10s, clean stage	55
4.32	Training curves for YOLOv10s, $\beta = 0.005$ stage	56
4.33	Training curves for YOLOv10s, $\beta = 0.01$ stage	56
4.34	Training curves for YOLOv10s, $\beta = 0.02$ stage	57

LIST OF ABBREVIATIONS

ADAS	Advanced Driver Assistance System(s)
AMP	Automatic Mixed Precision
AOD-Net	All-in-One Dehazing Network
AV	Autonomous Vehicle
CIB	Compact Inverted Block
CIoU	Complete Intersection over Union
CLAHE	Contrast Limited Adaptive Histogram Equalization
CNN	Convolutional Neural Network
COCO	Common Objects in Context (dataset)
CSPDarknet	Cross-Stage Partial Darknet
DFL	Distribution Focal Loss
DND-Net	Depth-Normalized Defogging Network
EHPD-Net	Efficient High-Precision Defogging Network
FLOPs	Floating-Point Operations
FPN	Feature Pyramid Network
FPS	Frames Per Second
GFLOPs	Giga Floating-Point Operations per Second
HOG	Histogram of Oriented Gradients
HR-YOLO 2025)	High-Robustness YOLO (fog-specific detector, Zhang & Jia,
IoU	Intersection over Union
ITS	Intelligent Transportation System(s)
mAP	mean Average Precision
mAP50	mean Average Precision at IoU threshold 0.5

mAP50-95	mean Average Precision averaged over IoU thresholds 0.5 to 0.95
NMS	Non-Maximum Suppression
PAN	Path Aggregation Network
PSA	Partial Self-Attention
R-CNN	Region-based Convolutional Neural Network
RTTS dataset)	Realistic Task-driven Testing Set (real-world foggy image
SCDown	Spatial-Channel Decoupled Downsampling
SMOTE	Synthetic Minority Over-sampling Technique
SPPF	Spatial Pyramid Pooling – Fast
SVM	Support Vector Machine
TOOD	Task-Aligned One-stage Object Detection
YOLO	You Only Look Once
Zero-DCE	Zero-Reference Deep Curve Estimation

ABSTRACT

Reliable vehicle detection in foggy conditions remains challenging for camera-based perception systems because fog degrades image contrast and object boundaries. This thesis evaluates YOLOv8s and a YOLOv10s-based preprocessing-and-training pipeline for five-class vehicle detection—car, truck, bus, motorcycle, and bicycle—under four sequential conditions: clear weather and three Foggy Cityscapes fog densities ($\beta=0.005$, 0.01 , and 0.02). Both models were trained using the same four-stage curriculum from clear to increasingly dense fog. The YOLOv10s pipeline additionally incorporated gamma correction, contrast-limited adaptive histogram equalization, and rare-class oversampling without modifying the detector architecture. The YOLOv10s pipeline achieved higher aggregate mAP50 than YOLOv8s across all conditions, including 0.587 versus 0.545 in clear weather. Under the heaviest fog, the models performed nearly identically, achieving 0.507 and 0.506 mAP50, respectively. Class-level results showed that YOLOv8s performed better for the rare truck and bus classes under heavy fog, demonstrating that aggregate metrics can conceal class-specific behavior. The YOLOv10s pipeline also exceeded the selected comparable HR-YOLO benchmark by approximately 1.2 percentage points under the heavy-fog setting. Overall, four-stage curriculum training and lightweight preprocessing improved foggy-scene vehicle detection, while class imbalance remained a major limitation for rare vehicle categories.

Keywords: *Adverse weather perception, class imbalance, curriculum training, deep learning, Foggy Cityscapes, object detection, vehicle detection, YOLOv8, YOLOv10*

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

Reliable vehicle detection is a core requirement for Intelligent Transportation Systems (ITS), underpinning applications from automated traffic monitoring to collision avoidance in autonomous and semi-autonomous vehicles [1]. Camera-based perception in particular has become central to the autonomous driving industry: companies including Tesla have moved toward camera-only perception stacks, arguing that vision alone, given sufficient data and model capacity, can support full self-driving without LiDAR [2]. Whether or not a purely camera-based approach ultimately proves sufficient, it is clear that camera-based object detection will remain a core component of autonomous and driver-assistance systems for the foreseeable future, which makes the robustness of that camera-based detection under real-world conditions, including adverse weather, a question of direct practical importance rather than only academic interest.

Camera-based object detectors have improved rapidly over the past decade, but this progress has been measured almost entirely on clear-weather benchmarks. Real deployment environments are not always clear: fog, in particular, is a common seasonal condition in many regions and degrades image contrast and edge definition in ways that clear-weather-trained detectors are not exposed to during training.

The Cityscapes dataset [3] and its synthetic-fog derivative, Foggy Cityscapes [4], have become standard benchmarks for studying this problem, since Foggy Cityscapes pairs each clear image with synthetically fogged versions at controlled severity levels while preserving the original annotations. This pairing makes it possible to isolate the effect of fog density on detection performance in a way that naturally collected foggy imagery, with its uncontrolled and unlabeled severity, does not allow.

Among detection architectures, the YOLO (You Only Look Once) family has become a preferred choice for real-time vehicle detection due to its favorable balance of accuracy

and inference speed. YOLOv8 [5] and YOLOv10 [6], the two most recent small-scale variants in this lineage at the time of this study, differ in several respects relevant to fog robustness: YOLOv10 in particular removes the Non-Maximum Suppression step required by YOLOv8 through a consistent dual-assignment training strategy, and introduces efficiency-oriented architectural changes intended to reduce computational redundancy. Whether these differences translate into a meaningful advantage specifically under fog, rather than only on the clear-weather benchmarks they were originally evaluated on, is a question the published literature has only partially answered.

1.2 Problem Statement

Prior work on fog-robust vehicle detection has consistently relied on architectural modification: adding a dedicated dehazing sub-network, an attention mechanism, or both, to a base detector. HR-YOLO [7], the strongest published supervised result on Foggy Cityscapes located during this study’s literature review, exemplifies this approach, combining a custom defogging backbone with a separate unsupervised dehazing network. This pattern leaves an open question: how much of the performance gained by these approaches comes from the added architectural complexity, and how much could be achieved through a curriculum-based training strategy on an unmodified architecture, combined only with lightweight, classical preprocessing rather than a learned dehazing module? A second, related gap is evaluative rather than architectural: most fog-detection studies report results at a single fog density, typically the moderate level, rather than characterizing how a model’s performance degrades across the full severity spectrum a real deployment might encounter. A third gap is specific to YOLOv10, which has seen very limited evaluation under controlled, multi-density fog conditions despite its favorable efficiency profile relative to YOLOv8. A fourth and final gap concerns class imbalance: vehicle datasets are heavily skewed toward common classes such as cars, and it is not well understood whether the vehicle classes most degraded by fog are also the classes least represented in training data, or whether the two effects are independent.

1.3 Research Motivation

The broader motivation for this research is a personal and long-term interest in contributing to autonomous vehicle technology of the kind pursued by companies such as Tesla, where

camera-based perception must operate reliably enough to drive a vehicle without human intervention. A driverless vehicle cannot choose the weather it operates in: it must detect other vehicles, pedestrians, and obstacles just as reliably in fog, rain, snow, or at night as it does in clear daylight. An official U.S. Department of Transportation study of automated vehicles under adverse weather found that fog limits a camera-based perception system’s ability to detect objects in essentially the same way it limits human vision, and documented significant, inconsistent performance degradation across multiple commercial Level 2 automated driving systems when tested under controlled adverse-weather conditions [8]. Independent sensor-evaluation research similarly reports measurable degradation in camera, radar, and LiDAR performance under rain, fog, and nighttime conditions [9]. These findings motivate the long-term goal behind this line of research: camera-based vehicle detection that remains dependable across the full range of adverse weather a real vehicle would encounter, not only fog, but also rain [10], snow [11], and low-light or nighttime conditions [12].

This thesis represents a first, deliberately scoped step toward that broader goal rather than a complete solution to it. It addresses fog specifically, since fog’s contrast-driven degradation is both a common real-world hazard and, through the Foggy Cityscapes benchmark, one of the few adverse-weather conditions with a controlled, multi-severity dataset available for systematic study (Section 1.1). Rain, snow, and nighttime conditions each degrade detection through different mechanisms (Section 2.7) and are left as future work (Section 5.6) rather than addressed here, so that this study could evaluate fog robustness thoroughly, across the full severity spectrum and two architectures, rather than superficially across many weather types at once. It is also important to be explicit about what this thesis does not claim: the models trained and evaluated here are tested on the Cityscapes and Foggy Cityscapes benchmark datasets, not on live video from an actual vehicle-mounted camera system, and the thesis addresses vehicle detection specifically rather than the full perception, prediction, and control stack a deployed autonomous vehicle would require. The contribution is a research step toward more weather-robust camera-based vehicle detection, not a deployable autonomous-driving camera system.

1.4 Research Objectives

1.4.1 General Objective

The general objective of this thesis is to systematically evaluate and compare the fog robustness of YOLOv8s and YOLOv10s for multi-class vehicle detection, using a curriculum-based training strategy across the full range of fog severities available in the Foggy Cityscapes benchmark.

1.4.2 Specific Objectives

To construct a reproducible annotation and training pipeline that converts Cityscapes polygon annotations to YOLO format and organizes clean and foggy imagery into a shared four-stage curriculum (Clean, $\beta = 0.005$, $\beta = 0.01$, $\beta = 0.02$).

To train YOLOv8s under a standard four-stage protocol, and YOLOv10s under the same curriculum augmented with a lightweight classical preprocessing pipeline (Gamma correction, CLAHE, and rare-class oversampling), without modifying either architecture.

To evaluate both models' detection performance, overall and per class, at each fog severity level, using Precision, Recall, mAP50, and mAP50-95.

To compare the two architectures quantitatively, qualitatively, and in terms of computational cost, and to situate the results against comparable published benchmarks.

To identify the extent to which class imbalance, rather than fog density alone, explains variation in per-class detection performance.

1.5 Research Questions

Does fog-density-specific curriculum fine-tuning improve multi-class vehicle detection robustness across fog severity levels, relative to a clean-trained baseline?

Under matched curriculum training, how does YOLOv10s's detection performance under fog compare to YOLOv8s's, both in aggregate and on a per-class basis?

Does a lightweight classical preprocessing pipeline, combined with curriculum training on an unmodified architecture, achieve fog-robustness gains comparable to published detectors that rely on dedicated dehazing sub-networks or attention mechanisms?

To what extent does class imbalance, rather than fog severity, explain which vehicle classes are detected least reliably under fog?

1.6 Scope of the Study

This study is scoped to synthetic fog simulation using the Foggy Cityscapes benchmark and does not involve real-world fog image collection or testing, nor deployment on a physical vehicle camera system; the implications of this choice are discussed as a limitation in Section 5.4. Evaluation is conducted on the Cityscapes and Foggy Cityscapes training and validation splits only. Five vehicle classes are considered: car, truck, bus, motorcycle, and bicycle. The study evaluates exactly two architectures, YOLOv8s and YOLOv10s, at the "small" model scale only; other YOLO generations, other model scales, and other detector families (such as transformer-based detectors) are outside its scope. As explained in Section 1.3 and established in Chapter 2 (Section 2.6), fog is the only adverse-weather condition investigated in this thesis; rain, snow, and low-light conditions, while part of the broader motivation for this line of research, are not evaluated here and are proposed as future work. Finally, the study does not modify either detector's architecture: all performance differences arise from training curriculum and preprocessing choices applied to otherwise unmodified, publicly available model implementations.

1.7 Significance of the Study

This study contributes to the object detection literature in four ways. First, it provides a systematic, multi-density evaluation of YOLOv10 under fog, a comparison largely absent from the published literature at the time of writing. Second, by deliberately keeping both architectures unmodified and isolating the effect of curriculum training and lightweight preprocessing, it offers evidence on how much of the fog-robustness gains reported by architecturally more complex prior work can be achieved without that added complexity. Third, its per-class analysis provides evidence on the relationship between class imbalance and fog robustness, a question with direct practical relevance for deploying vehicle

detection in regions where the classes most vulnerable to fog may also be under-represented in available training data. Fourth, at a practical level, the reproducible annotation and curriculum training pipeline developed for this study, along with its findings on computational cost (Section 4.2.2) and deployment-relevant trade-offs (Section 4.4.1), offers a concrete starting point for practitioners considering fog-aware vehicle detection for traffic monitoring, driver assistance, or, longer term, autonomous driving applications in fog-prone regions.

1.8 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 reviews the literature relevant to this study, covering the ITS and vehicle-detection context, the evolution of object detection architectures leading to the YOLO family, the specific challenge of fog degradation, classical and learned fog image-enhancement techniques, prior fog-specific detection work, domain-adaptation framings of the same problem, and class imbalance as an orthogonal challenge, concluding with the specific research gaps this thesis addresses. Chapter 3 describes the methodology, covering the datasets and annotation pipeline, the preprocessing steps applied to YOLOv10s training data, the model architectures, the four-stage curriculum training protocol for both models, and the evaluation configuration. Chapter 4 presents the results, covering quantitative and qualitative performance for both models, a detailed comparative analysis including computational efficiency, comparison against published benchmarks, and a discussion of the findings and their practical implications. Chapter 5 concludes the thesis, summarizing the study and revisiting the extent to which the research objectives were achieved, stating the major findings, discussing the limitations of the study, offering recommendations, outlining directions for future work, and closing with final remarks.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter reviews the literature underpinning this thesis, in a logical progression from broad context to specific gap. Sections 2.2–2.3 introduce Intelligent Transportation Systems and vehicle detection as the application context. Sections 2.4–2.6 review the general evolution of object detection and the YOLO family specifically. Section 2.7 examines adverse-weather detection challenges, with fog as the primary focus of this thesis and rain, snow, and low-light conditions covered for broader context. Section 2.8 reviews fog image-enhancement techniques, and Section 2.9 reviews prior fog-specific detection work directly relevant to this study. The chapter closes with the specific research gaps this thesis addresses (Section 2.10) and a brief summary (Section 2.11).

2.2 Intelligent Transportation Systems

Intelligent Transportation Systems (ITS) apply sensing, communication, and computation to improve the safety, efficiency, and management of road transportation. Camera-based visual perception is one of the most widely deployed sensing modalities within ITS, used for tasks including traffic flow monitoring, automated incident detection, and, in autonomous and semi-autonomous vehicles, real-time obstacle and vehicle detection. Ma and Xue [1] note that the reliability of these visual perception pipelines is a direct precondition for the broader goals of ITS, since downstream tasks such as traffic signal control, congestion prediction, and collision avoidance all depend on an accurate and timely stream of detected vehicles and obstacles. Weather-robust vehicle detection, the specific focus of this thesis, is therefore not only a computer vision problem but a prerequisite for dependable ITS operation in regions and seasons where fog is common.

2.3 Vehicle Detection

Vehicle detection is the task of localizing and classifying vehicle instances (typically by category, such as car, truck, bus, motorcycle, or bicycle) within an image or video frame. Ma and Xue [1] categorize vehicle detection approaches into three broad families: traditional handcrafted-feature methods (e.g., Histogram of Oriented Gradients combined

with a Support Vector Machine classifier), two-stage deep-learning detectors that first propose candidate regions and then classify them, and single-stage deep-learning detectors that predict object locations and classes directly in one forward pass. Handcrafted-feature methods have been largely superseded by deep-learning approaches over the past decade due to their limited robustness to appearance variation, including the appearance changes introduced by adverse weather. This thesis works exclusively with single-stage detectors from the YOLO family, motivated by their favorable accuracy-speed trade-off for real-time deployment, discussed further in Section 2.6.

2.4 Object Detection in Computer Vision

Modern object detection is dominated by deep convolutional neural network architectures, which can be broadly divided into two-stage and single-stage designs. Two-stage detectors originate with R-CNN, proposed by Girshick et al. [13], which generates candidate object regions using a separate region-proposal mechanism and then classifies each region with a convolutional network; while accurate, this design is computationally expensive since each region proposal requires an independent forward pass. Fast R-CNN [14] improved on this by sharing convolutional computation across all proposals within an image, and Faster R-CNN, proposed by Ren, He, Girshick, and Sun [15], replaced the external region-proposal step with a learned Region Proposal Network, making the entire pipeline trainable end-to-end and substantially faster. Despite these improvements, two-stage detectors remain slower than single-stage alternatives, which instead predict bounding boxes and class probabilities directly from a single dense pass over the image, trading a modest amount of accuracy for a substantial gain in inference speed — the design lineage from which the YOLO family (Section 2.5) descends.

Independent of weather robustness, class imbalance is a well-established challenge for object detectors trained on real-world driving datasets, where common classes (cars) vastly outnumber rare ones (motorcycles, bicycles, trucks, buses). Lin et al. [16] introduced Focal Loss to address foreground-background imbalance in single-stage detectors, showing that standard cross-entropy loss is dominated by easy negative examples. In the multi-class setting relevant to this thesis, image-level oversampling of rare-class examples, as formalized by Chawla et al. [17] for imbalanced classification generally, offers a model-

agnostic way to rebalance training exposure without altering the detector architecture — the approach adopted for the rare-class oversampling step described in Section 3.2.3.

2.5 Deep Learning for Object Detection

Zaidi et al. [18] survey the broader landscape of deep-learning-based object detection, characterizing the field along several axes: backbone feature-extraction design, the anchor-based versus anchor-free distinction in how candidate boxes are generated, and the loss functions used to jointly optimize classification and localization. They note that a persistent trend across the field has been the move toward anchor-free, single-stage designs that eliminate the manual tuning of anchor-box priors, a trend directly reflected in YOLOv8 and YOLOv10 (Sections 3.4.1–3.4.2), both of which use anchor-free detection heads. Zaidi et al. also emphasize that detector robustness under distribution shift, including but not limited to weather-induced shift, remains one of the least mature aspects of the field relative to raw benchmark accuracy on clean, curated datasets such as COCO — a gap this thesis addresses specifically for the fog sub-case.

2.6 YOLO Architecture

2.6.1 Evolution of YOLO

The YOLO (You Only Look Once) family, introduced by Redmon, Divvala, Girshick, and Farhadi [19], reframed object detection as a single regression problem, predicting bounding boxes and class probabilities directly from full images in one evaluation of the network, in contrast to the region-proposal pipelines described in Section 2.3. This design choice gave the original YOLO a substantial real-time speed advantage over contemporary two-stage detectors, at some initial cost to localization accuracy, particularly for small or densely packed objects. YOLO9000 (YOLOv2), proposed by Redmon and Farhadi [20], introduced batch normalization, anchor boxes adapted from Faster R-CNN, and a multi-scale training procedure that improved both accuracy and the range of input resolutions the network could handle. YOLOv3 [21] further improved small-object detection through multi-scale prediction at three different feature-map resolutions and a deeper backbone (Darknet-53). YOLOv4, introduced by Bochkovskiy, Wang, and Liao [22], consolidated a wide range of training and architectural refinements developed across the community

(including the CSPDarknet backbone, Mosaic data augmentation, and the CIoU loss used in this thesis; see Section 3.4.1) into a single cohesive design, establishing many of the conventions that subsequent YOLO generations, including YOLOv5 through YOLOv10, continue to build on. This thesis uses the two most recent small-scale variants in this lineage, YOLOv8s and YOLOv10s, described next.

2.6.2 YOLOv8

YOLOv8, released by Jocher, Chaurasia, and Qiu [5], introduced a decoupled, anchor-free detection head and the C2f feature-fusion module in place of the C3 module used in YOLOv5, offering a strong balance between accuracy and inference speed across a range of model scales. Chapter 3 (Section 3.4.1) describes YOLOv8s, the specific scale used in this thesis, in full architectural and mathematical detail.

2.6.3 YOLOv10

YOLOv10, introduced by Wang et al. [6] at NeurIPS 2024, extended the YOLOv8 design with a consistent dual-assignment training strategy that removes the need for Non-Maximum Suppression at inference time, alongside efficiency-oriented modules (SCDown, C2fCIB, PSA) that reduce computational redundancy relative to YOLOv8. Chapter 3 (Section 3.4.2) describes YOLOv10s in the same level of detail. This section and Section 2.8 instead review how each architecture has specifically been applied to fog-degraded detection in prior work.

2.7 Challenges of Vehicle Detection in Adverse Weather

2.7.1 Fog

Fog presents a uniquely difficult degradation for vision-based detection: unlike many other sources of image corruption, which tend to introduce spatially localized artifacts, fog degrades contrast and edge definition roughly uniformly across an entire scene, with severity that scales with scene depth according to the atmospheric scattering model. This depth dependence means that objects further from the camera are disproportionately affected, which is particularly consequential for vehicle detection, since distant vehicles are often the ones a driver or an automated system most needs advance warning about. The Foggy Cityscapes benchmark, introduced by Sakaridis, Dai, and Van Gool [4] and built on

the original Cityscapes dataset of Cordts et al. [3], simulates this depth-dependent degradation synthetically at three controlled severity levels and is the benchmark used throughout this thesis; both are described further in Section 3.1. Fog is the sole weather condition investigated in this thesis.

2.7.2 Rain

Rain degrades detection through mechanisms distinct from fog: individual raindrops and rain streaks introduce high-frequency visual noise, wet road surfaces produce specular reflections and glare, and heavy rain can reduce visibility in a manner similar to light fog. Hnewa and Radha [10] review object detection under rainy conditions specifically, and observe that convolutional detectors trained only on clear-weather data are particularly sensitive to the streak-like artifacts rain introduces, since these artifacts resemble high-frequency edge features the network may have learned to associate with object boundaries, leading to an elevated false-positive rate rather than the missed-detection pattern more commonly associated with fog. This distinction between rain’s artifact-driven degradation and fog’s contrast-driven degradation is one reason the two conditions are generally treated as separate research problems, and is part of the motivation (Section 1.3) for this thesis’s narrow current focus on fog alone.

2.7.3 Snow

Snow shares some visual characteristics with both rain (falling particulate matter) and fog (reduced overall visibility in heavy snowfall), while also introducing challenges specific to itself, such as snow accumulation altering the appearance of road surfaces and background clutter. Rjoub et al. [11] investigate vehicle and obstacle detection under snow using a YOLO-based detector combined with a federated learning framework across multiple vehicles, motivated by the observation that snow conditions vary substantially by geography and accumulation pattern, making a single centrally trained model less robust than one that can incorporate distributed, locally collected snow data. Snow is not evaluated in this thesis but is proposed as future work (Section 5.6), reinforcing that no single preprocessing or training strategy is likely to be uniformly optimal across all adverse-weather categories.

2.7.4 Low-Light Conditions

Low-light conditions, including nighttime and heavily overcast daytime scenes, degrade detection primarily through reduced signal-to-noise ratio and compressed dynamic range rather than through the scattering effects central to fog, rain, and snow. Guo et al. [12] propose Zero-Reference Deep Curve Estimation (Zero-DCE), a low-light image enhancement method that learns a pixel-wise curve mapping to improve visibility without requiring paired low-light and normal-light training data, an unsupervised design philosophy conceptually related to the domain-adaptation approaches discussed in Section 2.9.2. Although low-light conditions are outside the scope of this thesis (Section 1.6), they are included here for completeness, since fog scenes captured at dusk or under overcast skies often compound fog-induced contrast loss with reduced ambient illumination, a compounding effect not separately isolated in the Foggy Cityscapes benchmark used in this study.

2.8 Fog Image Enhancement Techniques

Image enhancement techniques applied prior to detection fall broadly into two categories: traditional, model-based methods that rely on an explicit physical or statistical model of fog formation, and learned, data-driven dehazing networks. Among traditional methods, the Dark Channel Prior, introduced by He, Sun, and Tang [23], observes that in most fog-free outdoor image patches, at least one color channel has some pixels with very low intensity, and uses the violation of this property to estimate fog transmission and recover a dehazed image without requiring any training data. Tarel and Hautiere [24] proposed a related fast visibility-restoration method based on median filtering that achieves similar qualitative results at substantially lower computational cost, making it suitable for real-time applications. Among learned approaches, Li et al. [25] proposed AOD-Net, a lightweight end-to-end trainable dehazing network that reformulates the atmospheric scattering model to allow direct image-to-image dehazing without separately estimating transmission and atmospheric light, and which has subsequently been used as a design basis for dehazing modules in several fog-specific detectors, including HR-YOLO (Section 2.8.3).

A distinct, non-physical-model approach relevant to this thesis is Contrast Limited Adaptive Histogram Equalization (CLAHE), introduced by Zuiderveld [26], which performs localized contrast enhancement without requiring a trained model or an explicit fog model, making it attractive for lightweight pipelines. However, the choice between a learned dehazing network and a classical, deterministic enhancement method is not merely a matter of computational convenience. Bıyık et al. [27] demonstrated at ICLR 2024 that deep dehazing networks optimized specifically to improve detection under fog can unexpectedly degrade detection performance on clear images, since such networks may learn transformations tailored to fog-specific artifacts that are counterproductive when no fog is present. Liu et al. [28] similarly argue that image restoration intended to support detection must be explicitly aligned with the detection objective rather than optimized for standalone perceptual quality, since the two objectives are not guaranteed to coincide. These findings directly motivate the choice made in this thesis (Section 3.2.2) to use classical, deterministic Gamma correction and CLAHE rather than a learned dehazing network for the YOLOv10s preprocessing pipeline, avoiding the risk of degrading clean-condition performance that a learned dehazing stage can introduce.

2.9 Related Works

2.9.1 Traditional Approaches

Prior to the widespread adoption of deep learning, fog-robust detection pipelines typically combined a traditional dehazing preprocessing step, such as the Dark Channel Prior [23] or the fast visibility-restoration method of Tarel and Hautiere [24], with a conventional handcrafted-feature detector such as HOG combined with an SVM classifier (Section 2.2). These pipelines were computationally inexpensive and required no fog-labeled training data, but generally underperformed learned approaches on both dehazing quality and downstream detection accuracy, and required manual parameter tuning that limited their robustness across different fog densities and scene types.

2.9.2 Deep Learning-Based Approaches

A substantial body of more recent work has adapted YOLO-family detectors specifically for foggy conditions, generally by inserting a defogging or enhancement stage, modifying

the backbone for better low-contrast feature extraction, or both. Cheng, Zhang, and Zheng [29] proposed a lightweight foggy-weather vehicle detector built on YOLOv8, replacing the standard C2f module with a dynamic-convolution variant (C2f-DCN) for improved adaptability to geometric distortion, and introducing an attention module (S5attention) to strengthen long-range feature dependencies; their preprocessing pipeline combined a learned dehazing network with classical dark-channel-prior enhancement before detection. Working with the earlier YOLOv5 architecture, Meng et al. [30] proposed YOLOv5s-Fog, which added a dedicated small-object detection layer and replaced standard Non-Maximum Suppression with Soft-NMS, reporting a 5.4-percentage-point mAP improvement over the YOLOv5s baseline on the real-world RTTS benchmark. Wang et al. [31] proposed R-YOLO, a detector aimed specifically at robustness across adverse-weather domains through a Quasi-Translation Network and a Feature Calibration Network that bridge the normal-to-adverse-weather domain gap. A common thread across this body of work is architectural modification: nearly every approach in this category adds a dedicated dehazing sub-network, a new attention mechanism, or both, to the base detector — an approach this thesis deliberately avoids in favor of curriculum training and lightweight preprocessing on an otherwise unmodified architecture (Section 2.10).

A related line of work addresses the same fog problem through domain adaptation rather than architectural modification. A parallel line of research addresses the clear-to-fog domain gap through unsupervised or semi-supervised domain adaptation, in which a detector is trained on labeled clear-weather data and unlabeled foggy data, without direct access to foggy ground-truth labels. Chen et al. [32] provide a general formulation of this domain-adaptive object detection problem for autonomous driving. Zhao et al. [33] proposed DAGL-Faster, a domain-adaptive Faster R-CNN incorporating graph-based feature alignment, reporting 36.49% mAP on Foggy Cityscapes. Wang et al. [34] proposed SS-WDA, which uses self-supervised contrastive learning to learn weather-robust features without foggy labels. Within the YOLO family specifically, Li et al. [35] proposed a Stepwise Domain-Adaptive YOLO (S-DAYOLO) framework that incrementally bridges the domain gap through an auxiliary intermediate domain, improving robustness under foggy conditions without directly training on labeled foggy data. These domain-adaptation results are consistently lower than supervised approaches trained and evaluated directly on

labeled foggy data (compare, for example, DAGL-Faster’s 36.49% mAP to HR-YOLO’s 49.5% mAP50, both on Foggy Cityscapes), which reflects the more difficult supervision setting rather than necessarily a weaker method; this distinction is treated carefully wherever domain-adaptation results are referenced quantitatively in Chapter 4.

2.9.3 Recent Studies (2023–2025)

Most directly relevant to this thesis is HR-YOLO, proposed by Zhang and Jia [7] in 2025, which combines a custom defogging backbone (EHPD-Net, incorporating a Swin Transformer block and a global-spatial attention module) with a separate lightweight unsupervised defogging network (DND-Net, built on the AOD-Net design of Li et al. [25]) applied as preprocessing, evaluated on both the real-world RTTS dataset and Foggy Cityscapes. HR-YOLO reports 79.8% mAP on RTTS and 49.5% mAP50 on Foggy Cityscapes at the heaviest fog density ($\beta = 0.02$), representing the strongest published supervised result on Foggy Cityscapes located during this review. Because it is evaluated under the same supervised, same-domain protocol used in this thesis rather than under domain adaptation, HR-YOLO is used directly as a comparison point in Chapter 4 (Section 4.3). Taken together with the 2023–2024 work reviewed in Section 2.9.2, the trajectory of recent fog-detection research shows a consistent pattern of incremental architectural sophistication — more elaborate attention mechanisms, dedicated defogging sub-networks, and hybrid CNN-Transformer backbones — with correspondingly modest accuracy gains on Foggy Cityscapes specifically, a pattern discussed further in Section 2.10.

Table 2.1 Key papers discussed in this section, by approach category.

Author	Model	Dataset	Metric	Year
He, Sun, and Tang	Dark Channel Prior (classical dehazing)	Custom outdoor hazy images	N/A	2010
Tarel and Hautière	Fast Visibility Restoration (classical)	Custom outdoor hazy images	N/A	2009

Li et al.	AOD-Net	Synthetic haze datasets	N/A	2017
Cheng, Zhang, and Zheng	Improved YOLOv8 (C2f-DCN + S5 attention)	RESIDE	+4.1 pp mAP50 vs YOLOv8	2024
Meng et al.	YOLOv5s-Fog	RTTS	73.40% mAP	2023
Meng et al.	YOLOv5s-Fog	Vn_ts	92.23% mAP	2023
Wang et al.	R-YOLO	Foggy Cityscapes	48.9% mAP	2022
Chen et al.	DA Faster	Foggy Cityscapes	27.6% mAP50	2018
Zhao et al.	DAGL-Faster	Foggy Cityscapes	36.49% mAP	2023
Hnewa and Radha	MS-DAYOLO	Foggy Cityscapes	41.5% mAP	2023
Li et al.	S-DAYOLO	Not verified	NR	2022
Zhang and Jia	HR-YOLO	RTTS / Foggy Cityscapes	79.8% mAP / 49.5% mAP50	2025

Note: N/A denotes that the metric was not applicable to the given study, while NR denotes that the value was not reported in the original publication.

2.10 Research Gap

Drawing the strands above together, four gaps recur across the reviewed literature. First, the great majority of fog-detection studies evaluate at a single fog density (most commonly $\beta = 0.01$), without characterizing how performance degrades across the full severity spectrum available in Foggy Cityscapes. Second, published fog-robust detectors almost universally rely on architectural modification — a custom dehazing sub-network, an added attention module, or both (Section 2.9.2–2.9.3) — leaving open the question of how much a curriculum-based training strategy on an unmodified architecture, combined with only lightweight classical preprocessing (Section 2.8), can achieve by comparison. Third, YOLOv10 specifically has seen very limited evaluation under controlled multi-density fog conditions despite its favorable efficiency profile relative to YOLOv8 (Section 2.6.3). Fourth, class imbalance (Section 2.4, above) and fog robustness are typically studied in

isolation, with few studies examining how the two interact — that is, whether the vehicle classes most degraded by fog are also the classes least represented in training data.

This thesis addresses these four gaps directly: it trains and evaluates both YOLOv8s and YOLOv10s across all three Foggy Cityscapes severity levels using a shared four-stage curriculum (Chapter 3, Sections 3.3.1–3.3.2); it deliberately keeps both architectures unmodified, isolating the contribution of curriculum training and lightweight preprocessing rather than architectural redesign; it provides one of the few controlled multi-density evaluations of YOLOv10 on Foggy Cityscapes located during this review; and its per-class analysis (Chapter 4, Section 4.4) explicitly examines the interaction between class imbalance and fog severity, finding that training-instance scarcity — not fog density — is the dominant factor separating strong- and weak-performing classes across both architectures.

2.11 Chapter Summary

This chapter reviewed the literature underpinning this thesis: the ITS and vehicle-detection context (Sections 2.2–2.3), the evolution of object detection from two-stage R-CNN variants through the anchor-free YOLO family (Sections 2.4–2.6), the specific challenge of fog degradation alongside rain, snow, and low-light conditions reviewed for broader context (Section 2.7), classical and learned fog image-enhancement techniques (Section 2.8), and prior fog-specific and domain-adaptation detection work, culminating in HR-YOLO as the closest comparable benchmark (Section 2.9). Four research gaps were identified in Section 2.10: limited multi-density fog evaluation, a reliance on architectural complexity in prior work, limited controlled evaluation of YOLOv10 under fog, and an unexamined interaction between class imbalance and fog severity. Chapter 3 now describes the methodology used to address these gaps.

CHAPTER 3

METHODOLOGY

This chapter describes the methodology used to address the research gaps identified in Chapter 2. the datasets and annotation pipeline (Section 3.1), the preprocessing steps applied to YOLOv10s training data (Section 3.2), the four-stage curriculum training protocol for both models (Section 3.3), the model architectures (Section 3.4), the hardware and training environment (Section 3.5), and the evaluation configuration (Section 3.6).

3.1 Dataset

This study utilizes two publicly available datasets: Cityscapes and Foggy Cityscapes. Cityscapes is a large-scale urban street scene dataset spanning 50 cities in Germany and neighboring countries, captured under clear-weather conditions [3]. The training images used in this study were drawn from 18 of these cities, while the validation images were drawn from a separate, non-overlapping set of cities, following the dataset's standard train/validation split. Cityscapes was used to provide the clean (fog-free) training and validation data. Foggy Cityscapes is a related benchmark dataset in which fog has been synthetically added to the original Cityscapes scenes using a depth-based rendering technique, and was used to provide the fog-affected training and validation data at three controlled density levels, defined by the attenuation coefficient β : $\beta = 0.005$ (light fog), $\beta = 0.01$ (moderate fog), and $\beta = 0.02$ (dense fog) [4].

Together, the clean Cityscapes images and the three fog density levels from Foggy Cityscapes formed four distinct visibility conditions, each used as a separate stage in the curriculum training pipeline. Figure 3.1 illustrates a representative scene across all four visibility conditions.

For each of the four conditions, the dataset was split into training and validation sets consisting of 2,975 and 500 images respectively, corresponding to an approximate 85:15 split. This resulted in a combined dataset of 11,900 training images and 2,000 validation images across all four stages.

Input image resolution differed between the two models evaluated in this study. For YOLOv8s, all images were resized to 640×640 pixels prior to training. For YOLOv10s, images were resized to 832×832 pixels, a higher resolution chosen to improve detection accuracy (mAP), particularly for small and distant objects that become harder to detect under fog.

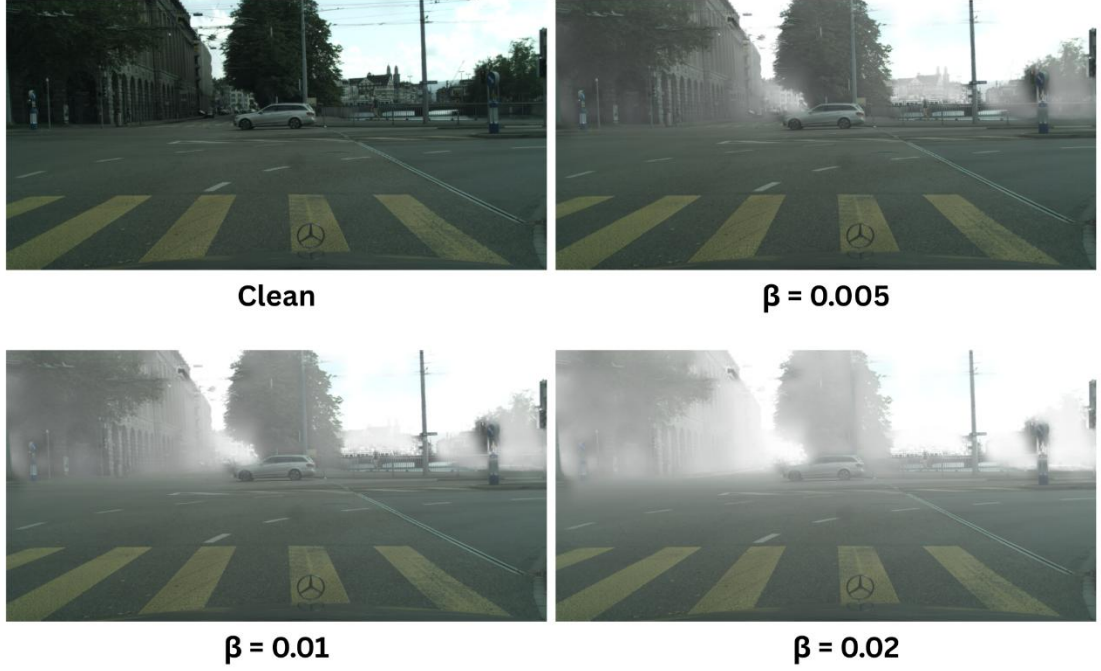


Figure 3.1 Sample scene shown at the four visibility conditions used across the curriculum stages: Clean, $\beta = 0.005$ (light fog), $\beta = 0.01$ (moderate fog), and $\beta = 0.02$ (dense fog).

3.1.1 Class Distribution

The five target vehicle classes are not represented equally in either the Cityscapes annotations or the resulting validation set. Table 3.1 reports the class-wise image and instance counts observed on the 500-image validation split (identical across all four visibility conditions, since Foggy Cityscapes inherits its annotations directly from Cityscapes).

Table 3.1 Class-wise image and instance counts on the validation split.

Class	Images Containing Class	Instances
Car	478	4,662
Bicycle	331	1,170
Motorcycle	93	149
Truck	80	93
Bus	75	98

Car instances outnumber the rarest class, truck, by a factor of exceeding 50 to 1, and bicycle is the only other class present in more than a third of validation images. This imbalance is present in the underlying Cityscapes annotations themselves rather than being an artifact of any processing step in this thesis, and motivates the rare-class oversampling strategy applied to motorcycle and bicycle during YOLOv10s training, described in Section 3.2.3. The same imbalance is referenced throughout the per-class analysis in Chapter 4 (Section 4.4), where it is identified as a more consistent predictor of per-class detection performance than fog density itself.

3.2 Preprocessing

Preprocessing was applied selectively to the YOLOv10s pipeline only; YOLOv8s was trained on the raw dataset without additional preprocessing, to serve as a baseline configuration. For YOLOv10s, three preprocessing steps were applied to each fog-density stage prior to training, described in the following subsections.

3.2.1 Annotation Cleaning

Bounding box annotations were first cleaned using an Intersection-over-Union (IoU) based deduplication procedure. For each image, boxes belonging to the same class with a pairwise IoU greater than 0.85 were considered duplicates, and only one instance from each overlapping cluster was retained.

3.2.2 Gamma Correction and CLAHE

Each image was corrected using a gamma value of 1.3, computed via a pixel intensity lookup table, to enhance overall visibility in foggy conditions. Contrast Limited Adaptive Histogram Equalization (CLAHE) was then applied to the L-channel of the image in LAB

color space, using a clip limit of 2.0 and a tile grid size of 8×8 , to locally enhance contrast without over-amplifying noise [26]. This lightweight, classical enhancement was preferred over learned dehazing networks, which have been shown to improve detection under fog at the cost of degrading performance on clear images, and which generally require explicit alignment with the detection objective rather than standalone perceptual-quality optimization [27], [28].



Figure 3.2 Effect of gamma correction ($\gamma = 1.3$) and CLAHE (clip limit = 2.0, tile size = 8×8) applied to a foggy ($\beta = 0.005$) training image.

3.2.3 Rare-Class Oversampling

To address class imbalance across the five target classes, images containing at least one instance of a rare class — motorcycle or bicycle — were identified from the converted YOLO-format annotation files and their image paths were listed a second time in the training image list, giving each such image an oversampling multiplier of $2\times$ relative to images containing only the more frequent classes (car, truck, bus).

It is important to be precise about what this oversampling does and does not do. No new images were generated, and no image files were duplicated or altered on disk: the underlying image and its annotation file remain a single physical file. The only change is at the sampling level — the image simply appears twice in the list the dataloader draws from for a given training epoch, so the model encounters it roughly twice as often as an image containing only frequent classes. Because the Ultralytics training pipeline applies its augmentation stack (mosaic composition, HSV colour jitter, horizontal flipping, and the gamma correction and CLAHE preprocessing described in Section 3.2.2) independently each time an image is drawn, the two appearances of a duplicated image are not identical: each is augmented separately, so the model effectively sees two different augmented views of the same underlying scene within an epoch rather than a literal repeated image. This is

a standard, low-cost way to rebalance training exposure toward under-represented classes without requiring synthetic data generation or a change to the detector architecture [16], [17].

3.3 Curriculum Training Pipeline

Both models were trained using a four-stage curriculum, in which each stage introduced a different visibility condition. The two models followed different stage orderings and hyperparameter configurations, described separately below. Figure 3.3 illustrates the training pipeline for both models, showing the checkpoint dependencies between stages.

3.3.1 YOLOv8s Training Configuration

YOLOv8s was trained across four stages in the order: Clean $\rightarrow \beta=0.01 \rightarrow \beta=0.005 \rightarrow \beta=0.02$. Stage 1 (Clean) was trained for 50 epochs, while Stages 2–4 were each trained for 30 epochs. All stages used a batch size of 8, an input resolution of 640×640 , an initial learning rate (lr0) of 0.001, automatic mixed precision (AMP) training, and an early-stopping patience of 10 epochs. The optimizer was left at Ultralytics' automatic selection setting (optimizer='auto'), and weight decay was left at the framework default of 0.0005. Stages 2, 3, and 4 were each fine-tuned independently from the Stage 1 (Clean) checkpoint, rather than sequentially from one another.

3.3.2 YOLOv10s Training Configuration

YOLOv10s was trained across four stages in the order: Clean $\rightarrow \beta=0.005 \rightarrow \beta=0.01 \rightarrow \beta=0.02$. All four stages used an input resolution of 832×832 , a batch size of 16, 50 epochs, an initial learning rate of 0.001, the AdamW optimizer, an early-stopping patience of 15 epochs, and AMP training. Mixup augmentation was set to 0.1 and mosaic augmentation to 1.0. Weight decay was left at the framework default of 0.0005. Unlike YOLOv8s, YOLOv10s followed a strictly sequential fine-tuning strategy, with each stage initialized from the immediately preceding stage's checkpoint (Stage 2 from Stage 1, Stage 3 from Stage 2, and Stage 4 from Stage 3).

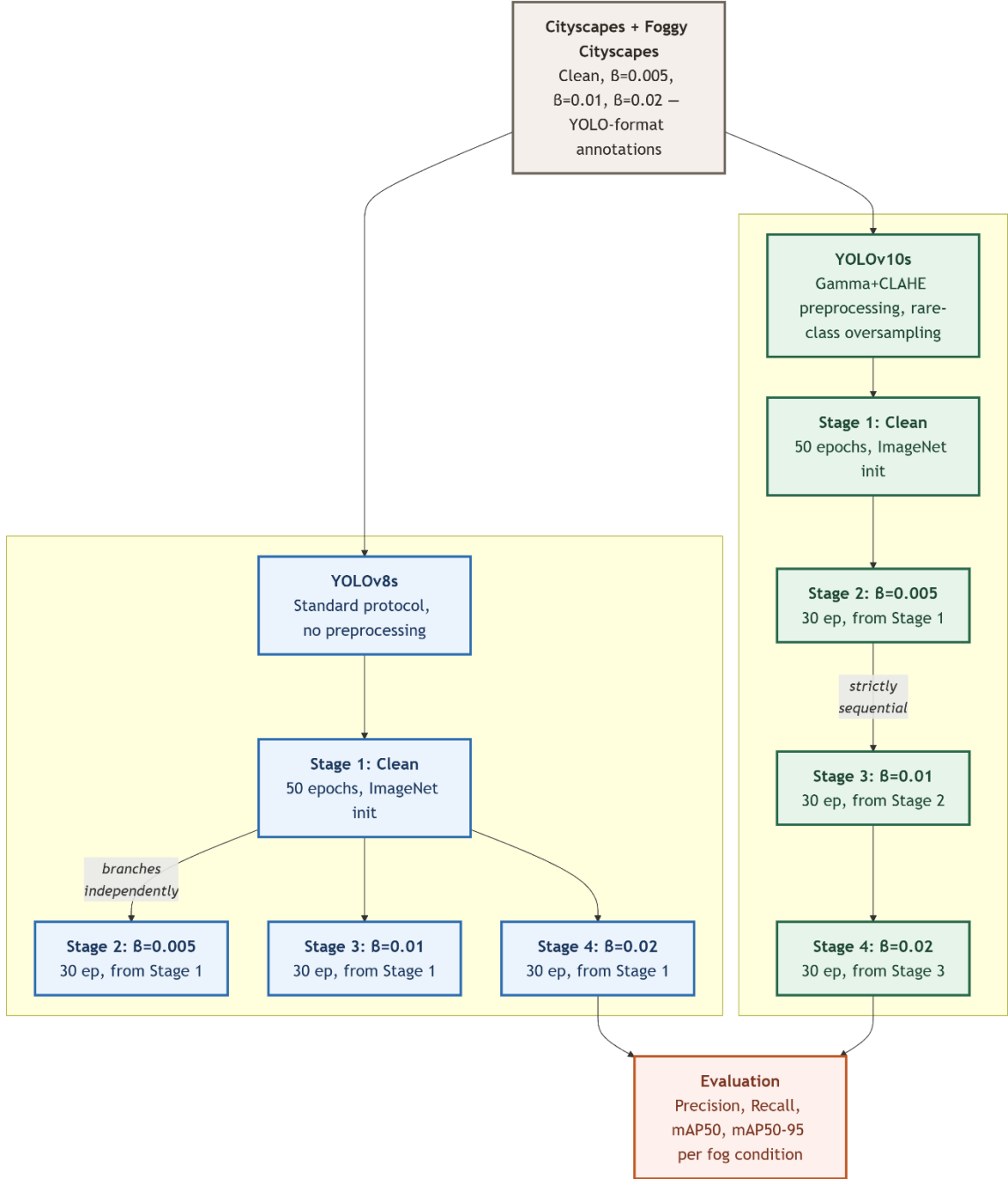


Figure 3.3 Detailed curriculum training pipeline for YOLOv8s and YOLOv10s. Both models train Stage 1 (Clean) identically from ImageNet-pretrained weights; YOLOv8s Stages 2–4 each fine-tune independently from the Stage 1 checkpoint, while YOLOv10s Stages 2–4 chain sequentially, each initialized from the previous fog stage. Only YOLOv10s receives the Gamma+CLAHE preprocessing and rare-class oversampling described in Section 3.2.

Table 3.2 Yolov8s Training hyperparameters.

Fog condition	Epoch	Batch	imgsz	Optimizer	lr0	Fine-tuned from
Clean	50	8	640	auto	0.001	Pretrained weights
$\beta = 0.01$	30	8	640	auto	0.001	Clean checkpoint
$\beta = 0.005$	30	8	640	auto	0.001	Clean checkpoint
$\beta = 0.02$	30	8	640	auto	0.001	Clean checkpoint

Table 3.3 Yolov10s Training hyperparameters.

Fog condition	Epoch	Batch	imgsz	Optimizer	lr0	Fine-tuned from
Clean	50	16	832	AdamW	0.001	pretrained weights
$\beta = 0.005$	50	16	832	AdamW	0.001	Clean checkpoint
$\beta = 0.01$	50	16	832	AdamW	0.001	$\beta = 0.005$ checkpoint
$\beta = 0.02$	50	16	832	AdamW	0.001	$\beta = 0.01$ checkpoint

3.4 Models

This study evaluates two object detection architectures from the YOLO (You Only Look Once) family: YOLOv8s and YOLOv10s, both representing the "small" variant within their respective model families, chosen to balance detection accuracy with computational efficiency suitable for real-time applications. Both models were implemented using the Ultralytics YOLO framework (version 8.4.90), which provides a unified API for training, validation, and inference across multiple YOLO architectures. No other model sizes (e.g., nano, medium, large, or extra-large variants) were evaluated in this study, as the small variants were selected to represent a practical trade-off between detection performance and deployment feasibility on resource-constrained hardware. The following two subsections describe each architecture in detail, including its layer-level structure and the mathematical

formulation of its core components, since both models share the same general convolutional-neural-network foundation but differ in several specific building blocks and in how the detection head is trained.

3.4.1 YOLOv8s Architecture

YOLOv8s follows the standard backbone–neck–head design used across the YOLO family [5]. At its base, every convolutional block in the network (referred to below as Conv) performs a 2D convolution followed by batch normalization and a SiLU (Sigmoid Linear Unit) activation:

$$y = SiLU(BN(W \cdot x + b)), SiLU(z) = z \cdot \sigma(z) \quad (3.1)$$

where x is the input feature map, W and b are the learnable convolution weights and bias, BN denotes batch normalization, and σ is the sigmoid function. All higher-level modules described below are built from this Conv unit.

The backbone is a CSPDarknet-based feature extractor arranged as an alternating sequence of strided Conv layers (which halve spatial resolution) and C2f (Cross-Stage Partial with two fused convolutions) blocks (which refine features at a fixed resolution). Table 3.4 lists the backbone layer sequence for the "small" (s) scale used in this study (depth multiple = 0.33, width multiple = 0.50, max channels = 1024), following the official Ultralytics YOLOv8 configuration, which reports 129 layers and 11,166,560 parameters at 28.8 GFLOPs for this scale at 640×640 input resolution [5].

Table 3.4 YOLOv8s backbone layer sequence ("s" scale).

Layer	Module	Stride level	Output channels	Output resolution (640×640 input)
0	Conv (3×3, s=2)	P1/2	32	320×320
1	Conv (3×3, s=2)	P2/4	64	160×160
2	C2f ×1	P2/4	64	160×160
3	Conv (3×3, s=2)	P3/8	128	80×80
4	C2f ×2	P3/8	128	80×80
5	Conv (3×3, s=2)	P4/16	256	40×40
6	C2f ×2	P4/16	256	40×40

7	Conv (3×3, s=2)	P5/32	512	20×20
8	C2f ×1	P5/32	512	20×20
9	SPPF (k=5)	P5/32	512	20×20

Within a C2f block, the input feature map is first passed through a 1×1 convolution and split channel-wise into two halves. One half is passed through a sequence of n Bottleneck sub-blocks (each two 3×3 convolutions with a residual connection), and, critically, the output of every Bottleneck is retained rather than only the final one. All of the intermediate outputs are concatenated together with the untouched half of the split and fused with a final 1×1 convolution:

$$X_1, X_2 = \text{split}(\text{Conv1x1}(X)); Y_i = \text{Bottleneck}(Y_{i-1}), Y_0 = X_2, i = 1..n;$$

$$\text{Out} = \text{Conv1x1}(\text{concat}(X_1, X_2, Y_1, \dots, Y_n)) \quad (3.2)$$

This design, in which $n+2$ feature maps are concatenated instead of only the last Bottleneck output as in the earlier C3 block, gives C2f a richer gradient flow and improves feature reuse without a proportional increase in parameter count.

At the end of the backbone, the Spatial Pyramid Pooling–Fast (SPPF) module aggregates multi-scale context by applying the same 5×5 max-pooling operation three times in sequence and concatenating all intermediate results:

$$y_1 = \text{MaxPool5x5}(x); y_2 = \text{MaxPool5x5}(y_1); y_3 = \text{MaxPool5x5}(y_2);$$

$$\text{Out} = \text{Conv1x1}(\text{concat}(x, y_1, y_2, y_3)) \quad (3.3)$$

This is functionally equivalent to the original Spatial Pyramid Pooling module’s multi-kernel design but considerably cheaper to compute, since it reuses a single small pooling kernel repeatedly instead of applying several differently sized kernels in parallel.

The neck combines a Feature Pyramid Network (FPN) with a Path Aggregation Network (PAN) structure: features from the P5 level are progressively upsampled and concatenated with earlier P4 and P3 backbone features (top-down path), and the resulting P3 features are then progressively downsampled and re-concatenated back up to P5 (bottom-up path), so that every output scale carries both high-level semantic information and fine-grained

spatial detail. Each concatenation point is followed by a C2f block (without the SPPF module) that fuses the merged features before passing them to the next stage.

The head is anchor-free and decoupled into two parallel branches operating on each of the three neck output scales (P3, P4, P5): a classification branch that predicts a per-class objectness score through an independent sigmoid activation for each class (a multi-label formulation, since a location can in principle belong to more than one class prediction before thresholding), and a regression branch that predicts the bounding box. Rather than regressing the four box edge distances directly as single scalars, YOLOv8s follows the Distribution Focal Loss (DFL) formulation: each of the four distances (left, top, right, bottom, relative to the grid cell) is predicted as a discrete probability distribution over 16 integer bins ($\text{reg_max} = 16$), and the final distance is recovered as the expectation of that distribution [36]:

$$\hat{d} = \sum_{i=0}^{15} P(i) \cdot i \quad (3.4)$$

where $P(i)$ is the predicted probability assigned to bin i . This allows the model to express uncertainty about ambiguous box edges (for example, occluded or fog-obscured boundaries) as a distribution rather than forcing a single, potentially over-confident point estimate.

Training combines three loss terms: a binary cross-entropy loss for classification, a Complete Intersection over Union (CIoU) loss for the overall box quality, and the DFL loss for the distributional box regression described above. The CIoU loss extends standard IoU loss with a penalty on the normalized distance between box centers and a consistency term on aspect ratio [37]:

$$CIoU = IoU - \frac{\rho^2(b, b_{GT})}{c^2} - \alpha v \quad (3.5)$$

$$\alpha = \frac{v}{1 - IoU + v} \quad (3.6)$$

$$v = \left(\frac{4}{\pi^2}\right) \cdot \left(\arctan\left(\frac{w_{GT}}{h_{GT}}\right) - \arctan\left(\frac{w}{h}\right)\right) \quad (3.7)$$

where $\rho(b, b_{GT})$ is the Euclidean distance between the predicted box center b and the ground-truth box center b_{GT} , c is the diagonal length of the smallest enclosing box

containing both, and w , h and wgt , hgt are the predicted and ground-truth box widths and heights respectively. The total training loss is a weighted sum of the three terms, $L = \lambda_{box} \cdot (1 - CIoU) + \lambda_{cls} \cdot LBCE + \lambda_{dfl} \cdot LDFL$, with the default Ultralytics weighting used throughout this study.

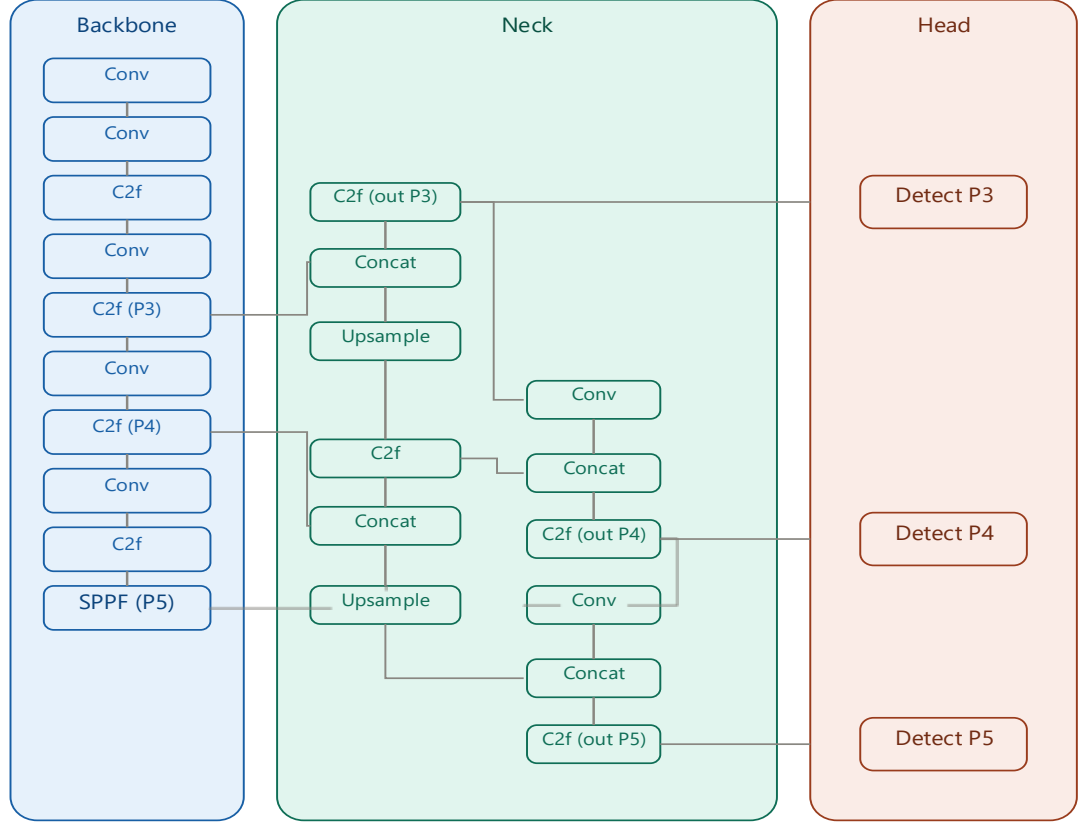


Figure 3.4 YOLOv8s network architecture, showing the backbone, neck, and multi-scale detection head (P3, P4, P5 outputs).

3.4.2 YOLOv10s Architecture

YOLOv10s retains the same Conv, C2f, and SPPF building blocks described in Section 3.4.1 for the earlier backbone stages, but introduces three additional modules aimed at removing the need for Non-Maximum Suppression (NMS) during inference and reducing computational redundancy [6]. Table 3.5 lists the backbone layer sequence and module types, following the structure of the official YOLOv10 configuration.

Table 3.5 YOLOv10s backbone layer sequence and module types.

Layer	Module	Stride level	Role
0	Conv (3×3, s=2)	P1/2	Stem
1	Conv (3×3, s=2)	P2/4	Downsample
2	C2f	P2/4	Feature refinement
3	Conv (3×3, s=2)	P3/8	Downsample
4	C2f	P3/8	Feature refinement
5	SCDown	P4/16	Efficient downsample
6	C2f	P4/16	Feature refinement
7	SCDown	P5/32	Efficient downsample
8	C2fCIB	P5/32	Lightweight feature refinement
9	SPPF (k=5)	P5/32	Multi-scale context aggregation
10	PSA	P5/32	Global self-attention

At P4/16 and P5/32, the standard strided Conv used in YOLOv8s is replaced with Spatial-Channel Decoupled Downsampling (SCDown), which separates channel-dimension change from spatial-resolution reduction into two sequential, cheaper operations rather than performing both simultaneously in a single standard convolution:

$$SCDown(X) = DWConv_{3\times3,s=2}(PWConv_{1\times1}(X)) \quad (3.8)$$

A pointwise (1×1) convolution first adjusts the channel dimension, and a depthwise 3×3 convolution with stride 2 then performs the spatial downsampling independently per channel. Because the depthwise convolution operates on each channel separately rather than mixing all input channels together as a standard convolution does, this decoupling substantially reduces the parameter count and computation of the downsampling step while preserving the same output shape.

At the final backbone stage, the C2f block is replaced with C2fCIB, which substitutes the internal Bottleneck sub-blocks with Compact Inverted Blocks (CIB): a depthwise 3×3 convolution handles spatial mixing and a pointwise 1×1 convolution handles cross-channel mixing, following the same depthwise/pointwise decomposition principle as SCDown but

applied to feature refinement rather than downsampling. This keeps the deepest, most channel-heavy stage of the network inexpensive to compute.

After SPPF, YOLOv10s adds a Partial Self-Attention (PSA) block, in which the feature map is split in half; one half is passed through a standard multi-head self-attention operation followed by a feed-forward layer, and the other half bypasses attention entirely and is concatenated back afterward. The self-attention operation itself follows the standard scaled dot-product formulation [38]:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}}) \cdot V \quad (3.9)$$

where Q, K, and V are the query, key, and value projections of the input features and d_k is the key dimension used to scale the dot product. Applying attention to only half of the channels, rather than the full feature map, gives PSA most of the benefit of global context modelling — which is useful for relating spatially distant but related object parts, and for aggregating faint fog-degraded cues across a wider receptive field — at a fraction of the computational cost of full self-attention. The neck retains the same FPN+PAN structure described in Section 3.4.1.

The most significant change relative to YOLOv8s is in how the detection head is trained. YOLOv10s keeps the same anchor-free, decoupled, DFL-based head design described in Section 3.4.1 (Equations 4–7 apply unchanged), but adds a second, parallel one-to-one head trained alongside the original one-to-many head, a strategy referred to as consistent dual assignment [6]. Both heads are trained to agree with the same alignment metric, which jointly scores how well a predicted box matches a ground-truth object in terms of both classification confidence and localization quality:

$$m(\alpha, \beta) = s^\alpha \cdot u^\beta \quad (3.10)$$

where s is the predicted classification score, u is the Intersection over Union between the predicted and ground-truth boxes, and α and β are hyperparameters balancing the relative importance of classification confidence versus localization accuracy (following the task-alignment formulation used in one-to-many label assignment for modern anchor-free detectors [39]). The one-to-many head uses this metric to assign multiple positive samples

per ground-truth object during training, providing the rich supervisory signal that anchor-free detectors typically rely on. The one-to-one head uses the same metric but keeps only the single highest-scoring match per ground-truth object, so it is trained to predict exactly one confident box per object rather than several overlapping candidates. Because both branches share the backbone and neck and are optimized jointly, the one-to-one head inherits the strong feature representations learned under the one-to-many head’s denser supervision. At inference time, only the one-to-one head is used, and since it has been trained specifically to produce a single prediction per object, the Non-Maximum Suppression post-processing step required by YOLOv8s and earlier YOLO versions becomes unnecessary. Published results report that this design gives YOLOv10s approximately 7.2 million parameters and 21.6 GFLOPs at 640×640 input resolution, roughly 35% fewer parameters than YOLOv8s at a comparable accuracy level on general benchmarks [6].

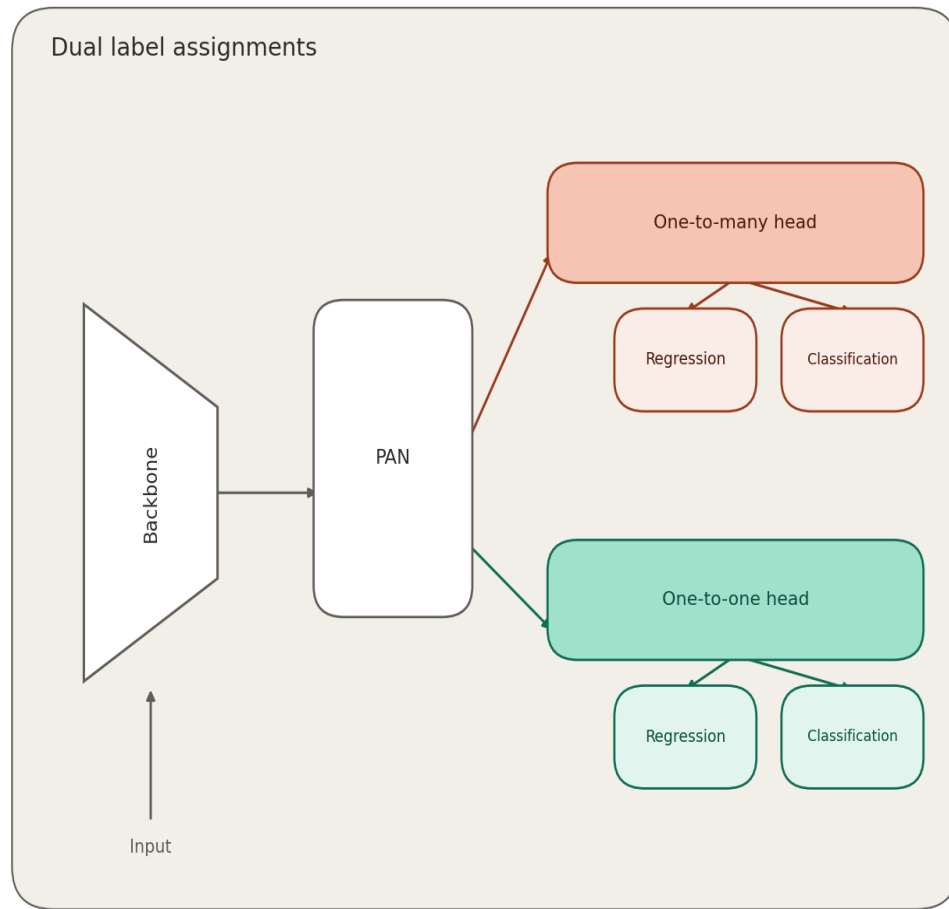


Figure 3.5 Illustration of the consistent dual assignment strategy: a shared backbone and PAN neck feed two heads, namely the one-to-many head and the one-to-one head, with each head performing classification and regression.

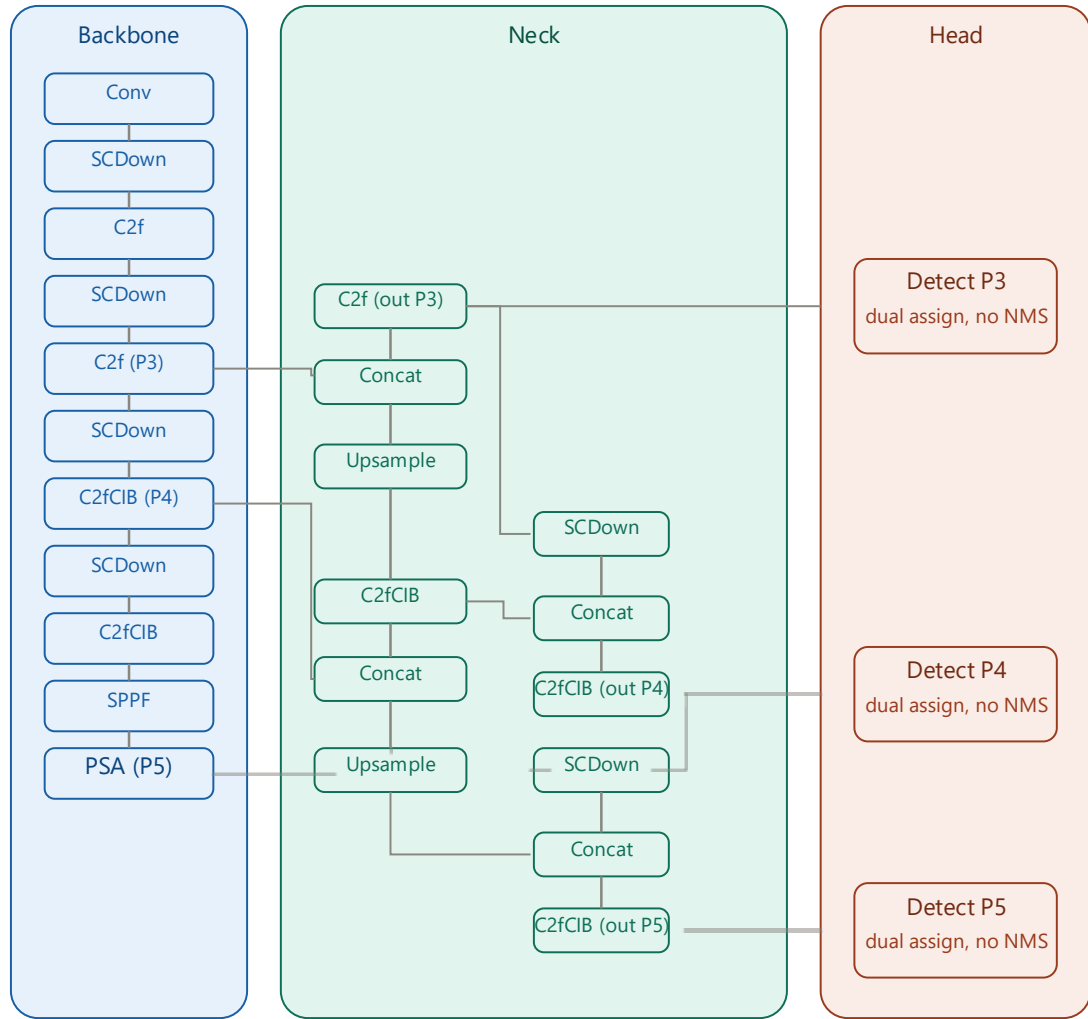


Figure 3.6 YOLOv10s network architecture, showing the backbone with SCDown and C2fCIB modules, the PSA block, and the dual-assignment detection head (P3, P4, P5 outputs).

3.5 Hardware/Environment

All training was conducted on Google Colab using the free tier, with an NVIDIA T4 GPU. Due to the session time limits imposed on the free tier, training was periodically interrupted; in such cases, training was resumed from the last saved checkpoint (last.pt) across multiple Colab sessions to continue the training process without loss of progress. No other specialized hardware or environment configurations were used.

3.6 Evaluation

3.6.1 Evaluation Metrics

Model performance was evaluated using four standard object detection metrics: Precision, Recall, mean Average Precision at an IoU threshold of 0.5 (mAP50), and mean Average Precision averaged across IoU thresholds from 0.5 to 0.95 (mAP50-95). These metrics are consistent with those commonly reported in the object detection literature and allow direct comparison with prior fog-detection studies [7].

3.6.2 Evaluation Protocol

Each trained model was evaluated on the validation set corresponding to its training condition: models trained on clean images were evaluated on the clean validation set, while models trained under a given fog density were evaluated on the validation set of the same fog density. This per-condition evaluation protocol reflects the curriculum training design, in which each stage is intended to specialize the model for a specific visibility condition rather than for generalized cross-condition robustness.

3.6.3 Best-Epoch Selection

For each training run, the reported model checkpoint was selected using Ultralytics' fitness-based convention rather than the final epoch, since validation performance can fluctuate across epochs. This fitness score is computed as a weighted combination of mAP50-95 (90%) and mAP50 (10%), and the checkpoint achieving the highest fitness score across all epochs of a given stage was selected as that stage's best-performing model (best.pt). This ensures that reported results reflect each model's peak validation performance rather than an arbitrary stopping point.

This chapter described the datasets, preprocessing pipeline, curriculum training protocol, model architectures, hardware environment, and evaluation configuration used in this study. Chapter 4 now presents the results produced by this methodology.

CHAPTER 4

RESULTS AND DISCUSSION

This chapter presents the results produced by the methodology described in Chapter 3. Section 4.1 reports quantitative and qualitative results for both models. Section 4.2 provides a detailed comparative analysis, including computational efficiency. Section 4.3 compares the results against published benchmarks. Section 4.4 discusses the findings and their practical implications.

4.1 Results

This section presents the quantitative performance of both models across all four curriculum stages. Metrics were extracted from the fitness-weighted best epoch of each training run, as described in Section 3.6.3. Precision, Recall, mAP50, and mAP50-95 are reported for each stage, followed by per-class mAP50 breakdowns where available.

4.1.1 YOLOv8s Results

Table 4.1 presents the overall detection performance of YOLOv8s across its four curriculum stages.

Table 4.1 YOLOv8s overall performance by stage.

Stage	Precision	Recall	mAP50	mAP50-95
Clean	0.696	0.483	0.545	0.345
$\beta = 0.01$	0.698	0.457	0.514	0.321
$\beta = 0.005$	0.732	0.428	0.511	0.326
$\beta = 0.02$	0.667	0.458	0.506	0.319

Performance declines from the clean baseline to all three foggy conditions, with mAP50 dropping by 3.1 to 3.9 percentage points depending on fog density. Notably, the three foggy stages score within a narrow 0.8-point mAP50 band of each other (0.506–0.514), despite

differing fog densities and differing training-stage orders (Clean $\rightarrow \beta 0.01 \rightarrow \beta 0.005 \rightarrow \beta 0.02$). This suggests that once YOLOv8s is fine-tuned away from the clean baseline, its performance is relatively insensitive to the specific fog density it was fine-tuned on.

Table 4.2 breaks these results down by class.

Table 4.2 YOLOv8s per-class mAP50 by stage.

Class	Clean	$\beta = 0.01$	$\beta = 0.005$	$\beta = 0.02$
Car	0.749	0.717	0.735	0.713
Truck	0.406	0.367	0.313	0.378
Bus	0.679	0.614	0.642	0.582
Motorcycle	0.408	0.398	0.385	0.377
Bicycle	0.482	0.474	0.483	0.477

Figures 4.1–4.4 present the normalized confusion matrices for YOLOv8s for each of the four visibility conditions individually, and Figures 4.5–4.8 present the corresponding precision-recall curves for each condition.

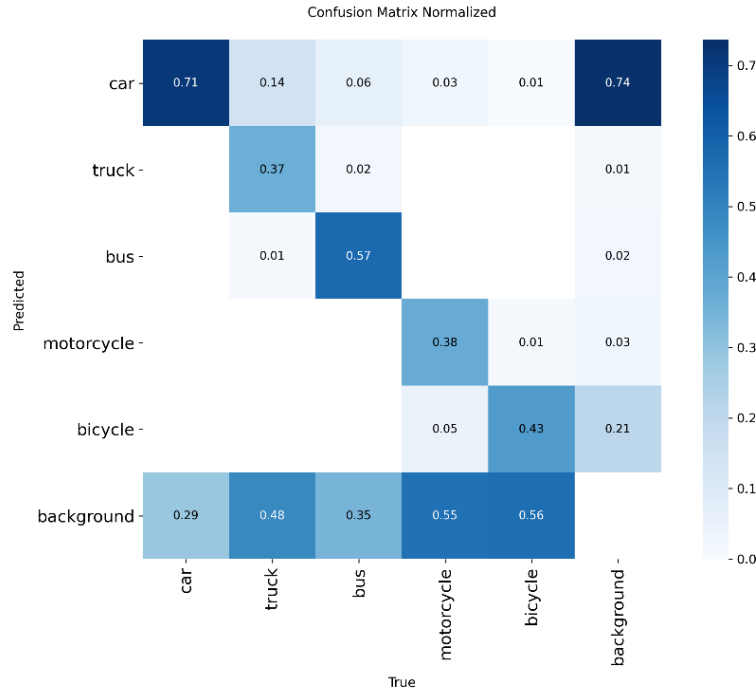


Figure 4.1 Normalized confusion matrix for YOLOv8s, Clean condition.

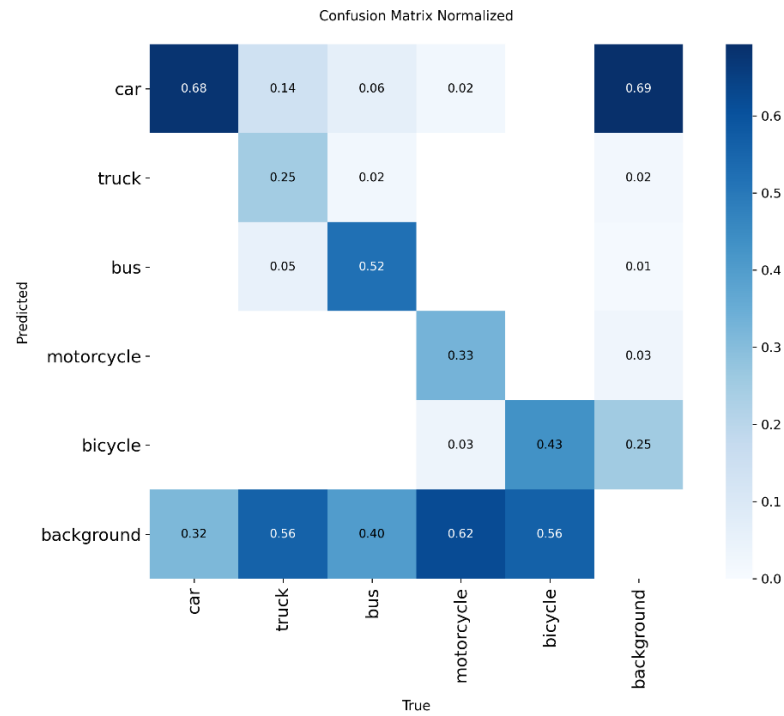


Figure 4.2 Normalized confusion matrix for YOLOv8s, $\beta = 0.005$ condition.

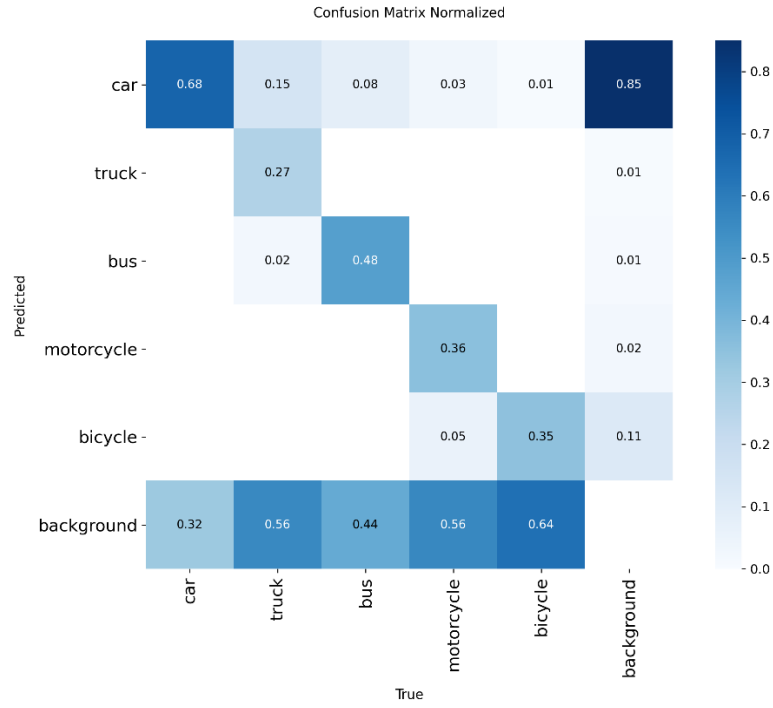


Figure 4.3 Normalized confusion matrix for YOLOv8s, $\beta = 0.01$ condition.

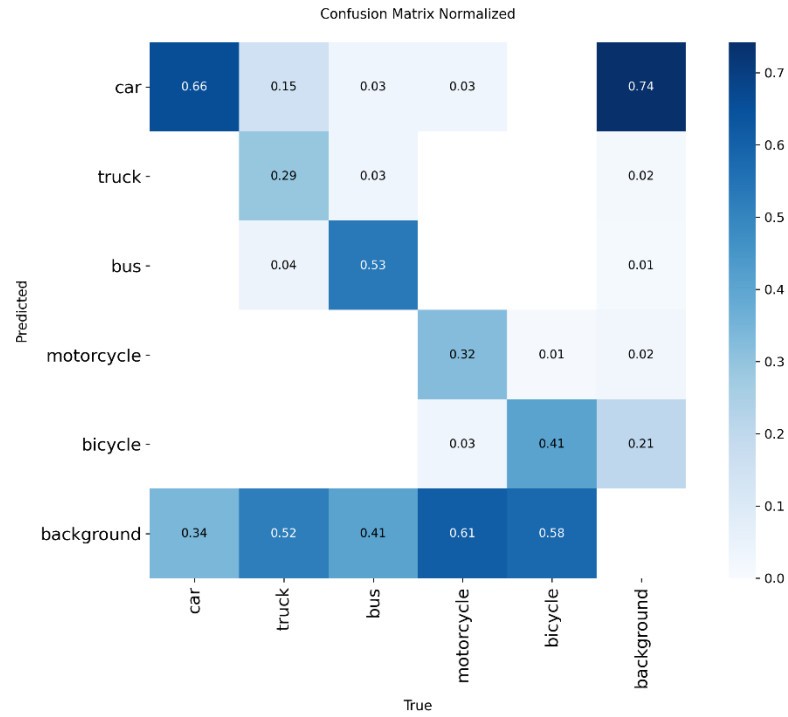


Figure 4.4 Normalized confusion matrix for YOLOv8s, $\beta = 0.02$ condition.

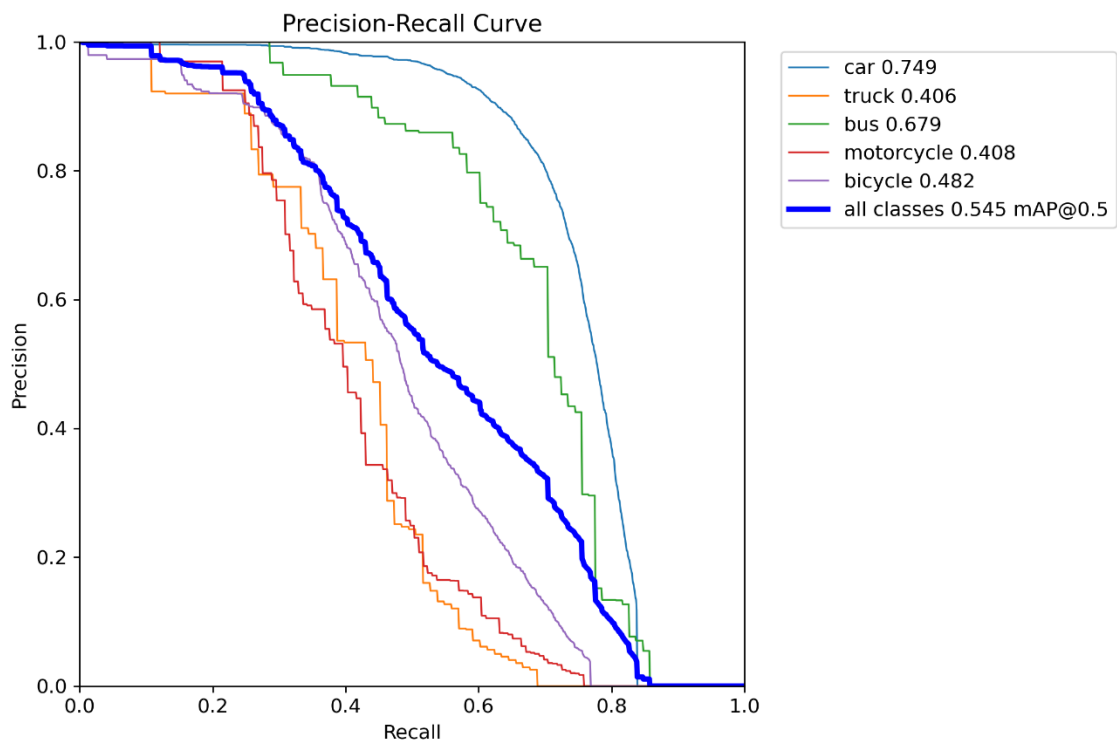


Figure 4.5 Precision-recall curve for YOLOv8s, Clean condition.

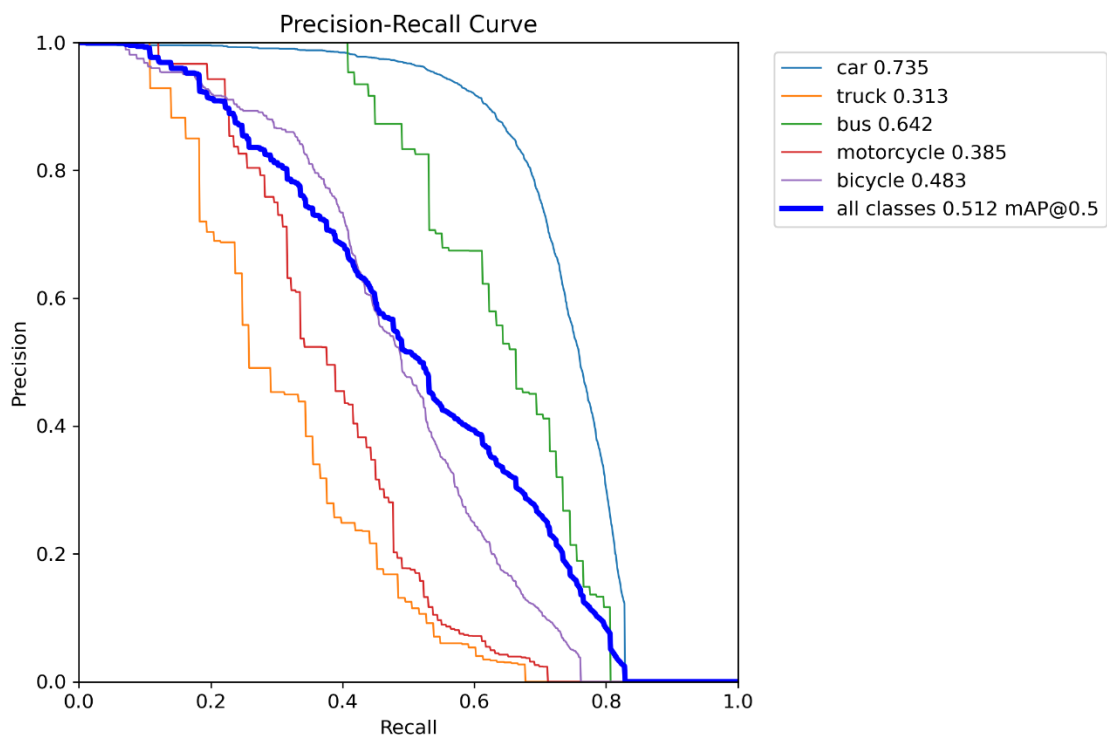


Figure 4.6 Precision-recall curve for YOLOv8s, $\beta = 0.005$ condition.

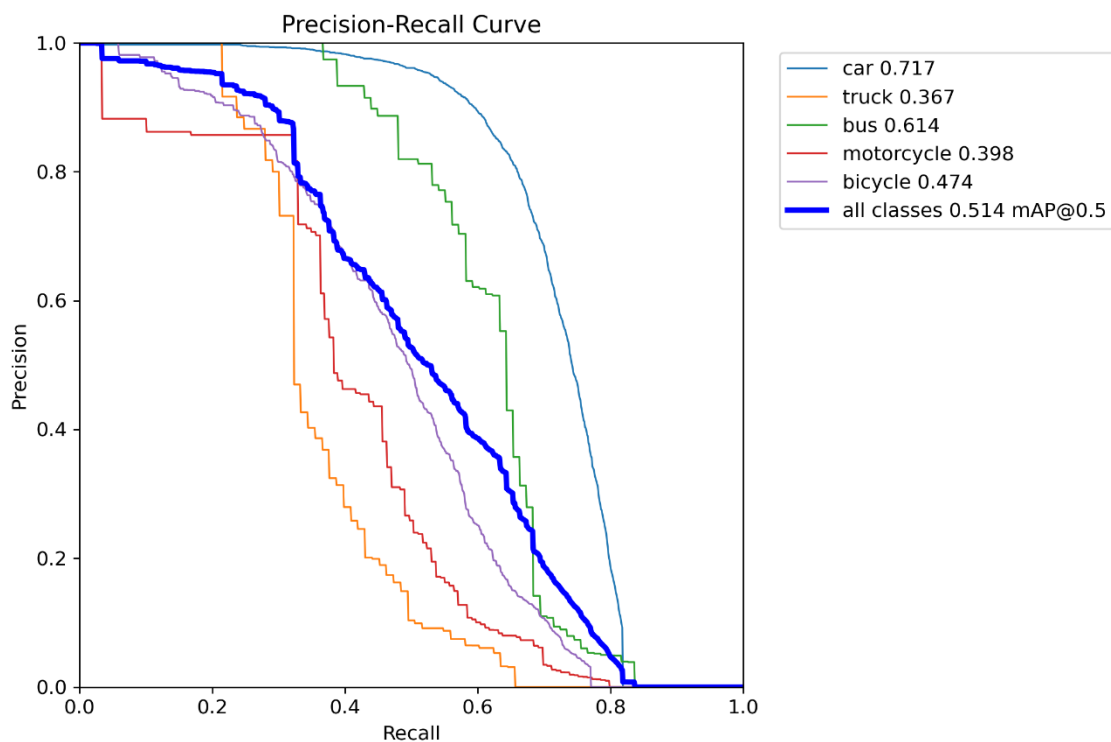


Figure 4.7 Precision-recall curve for YOLOv8s, $\beta = 0.01$ condition.

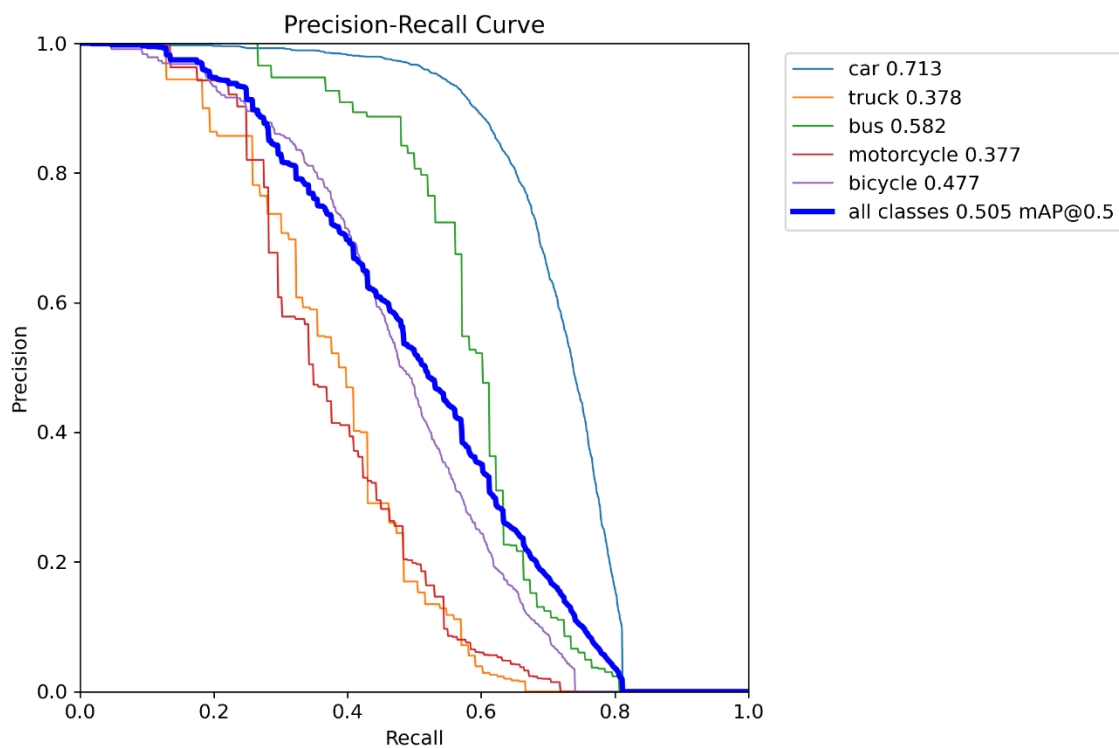


Figure 4.8 Precision-recall curve for YOLOv8s, $\beta = 0.02$ condition.

The confusion matrices in Figures 4.1–4.4 show a consistent pattern across all four conditions: car is the only class with a strong, stable diagonal (0.66–0.71 across stages), while every other class is confused with the background category more often than with any other vehicle class, and background is itself the largest single source of false predictions for every class. This pattern changes little between Clean and $\beta = 0.02$, indicating that the dominant error mode for YOLOv8s is a general tendency to miss or misattribute low-frequency classes to background rather than a fog-specific confusion between visually similar vehicle types (e.g., car/truck or bicycle/motorcycle). The precision-recall curves in Figures 4.5–4.8 show the same car-versus-rest gap directly: the car curve remains close to the top-right corner at every fog density, while truck and motorcycle collapse toward the origin fastest, consistent with the low instance counts for these classes reported in Table 3.1.

Across both conditions, car achieves the highest per-class mAP50, followed by bus, bicycle, motorcycle, and truck. This ordering is consistent with the number of training instances available per class (4,662 car instances versus 93 truck and 98 bus instances in the validation set), suggesting that class imbalance, rather than fog alone, is a major contributor to the lower-performing classes.

4.1.2 YOLOv10s Results

Table 4.3 presents the overall detection performance of YOLOv10s across its four curriculum stages.

Table 4.3 YOLOv10s overall performance by stage.

Stage	Precision	Recall	mAP50	mAP50-95
Clean	0.723	0.524	0.587	0.385
$\beta = 0.005$	0.735	0.504	0.568	0.372
$\beta = 0.01$	0.779	0.456	0.535	0.347
$\beta = 0.02$	0.762	0.455	0.507	0.325

Unlike YOLOv8s, YOLOv10s shows a clear, monotonic decline in mAP50 as fog density increases ($0.587 \rightarrow 0.568 \rightarrow 0.535 \rightarrow 0.507$), tracking the physical difficulty progression of the curriculum design. This is consistent with YOLOv10s's sequential fine-tuning strategy (each stage initialized from the previous stage's checkpoint, as described in Section 3.3.2), which directly mirrors the increasing fog density order.

Table 4.4 presents the per-class breakdown across all four conditions.

Table 4.4 YOLOv10s per-class mAP50 by stage.

Class	Clean	$\beta = 0.005$	$\beta = 0.01$	$\beta = 0.02$
Car	0.805	0.789	0.785	0.757
Truck	0.460	0.408	0.385	0.341
Bus	0.684	0.646	0.585	0.568
Motorcycle	0.437	0.429	0.430	0.384
Bicycle	0.548	0.529	0.491	0.485

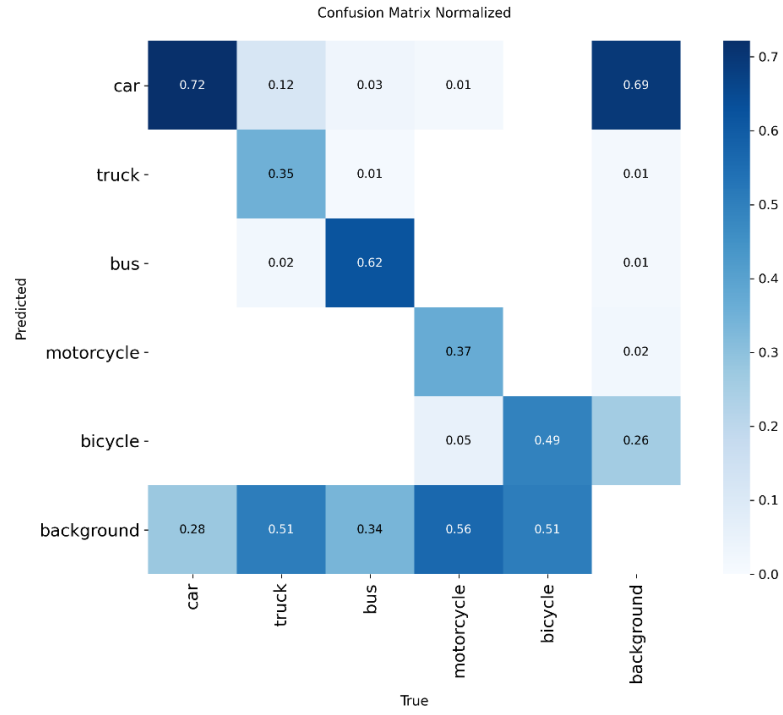


Figure 4.9 Normalized confusion matrix for YOLOv10s, Clean condition.

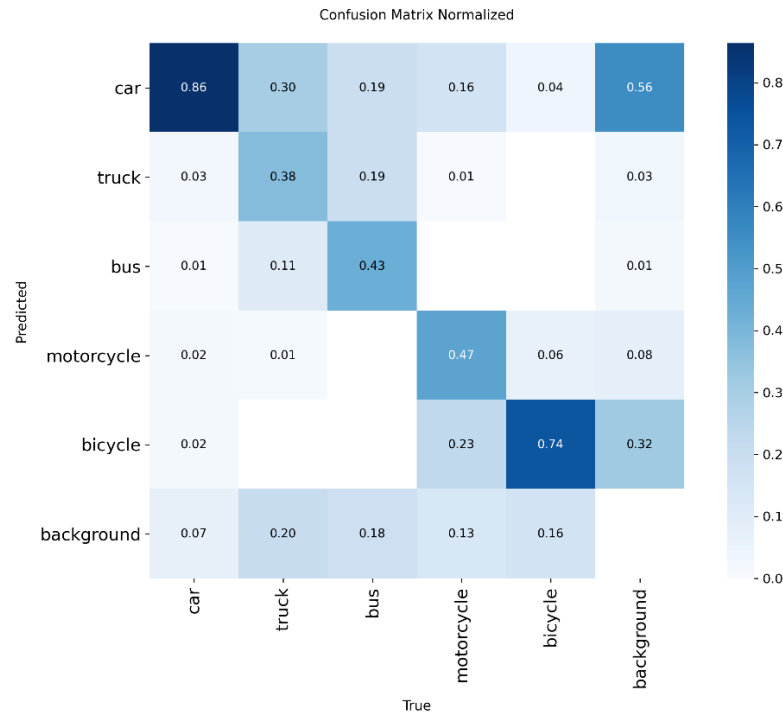


Figure 4.10 Normalized confusion matrix for YOLOv10s, $\beta = 0.005$ condition.

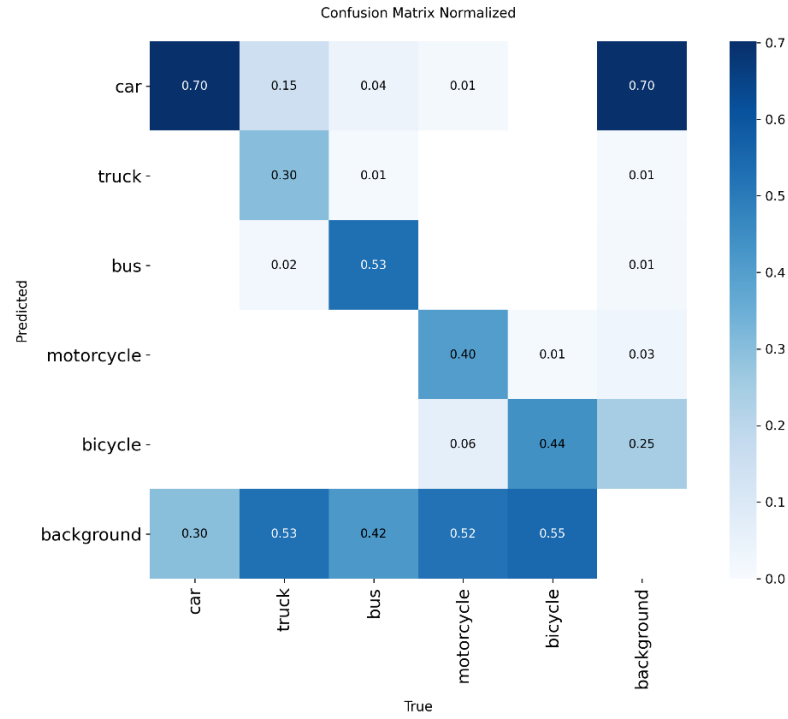


Figure 4.11 Normalized confusion matrix for YOLOv10s, $\beta = 0.01$ condition.

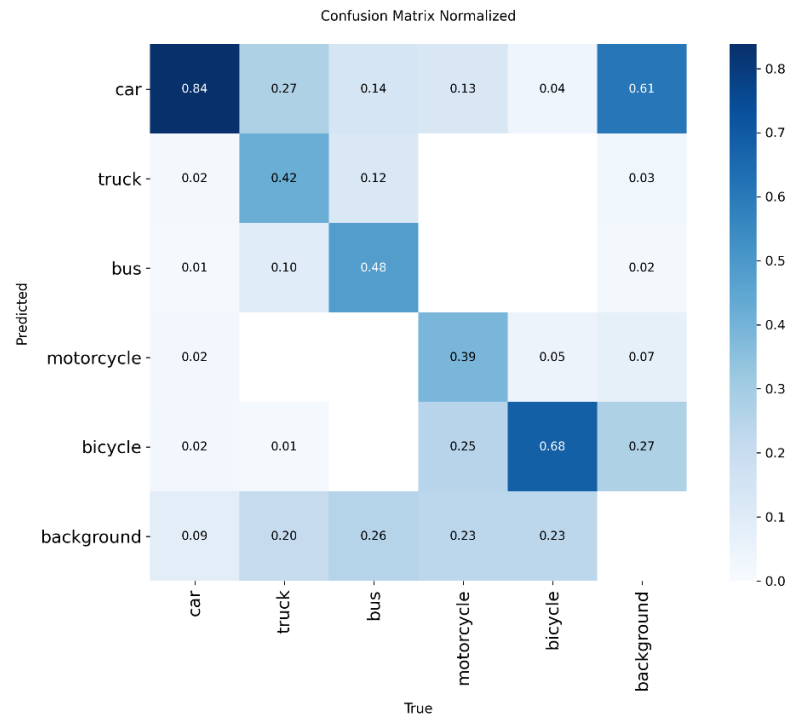


Figure 4.12 Normalized confusion matrix for YOLOv10s, $\beta = 0.02$ condition.

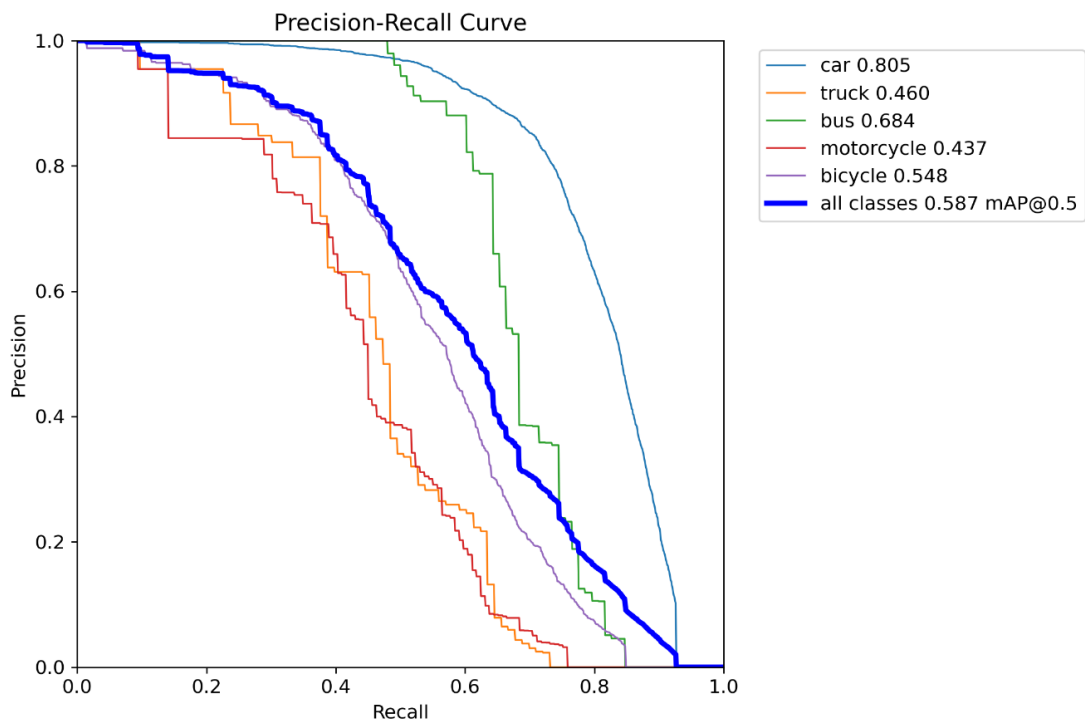


Figure 4.13 Precision-recall curve for YOLOv10s, Clean condition.

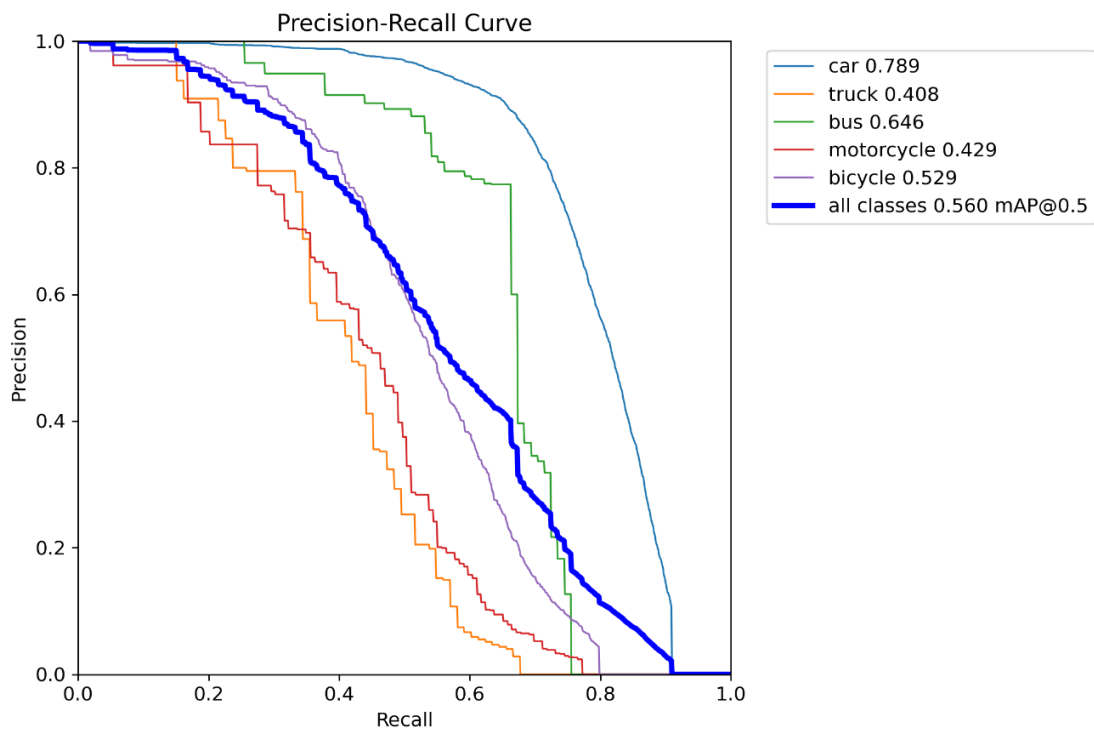


Figure 4.14 Precision-recall curve for YOLOv10s, $\beta = 0.005$ condition.

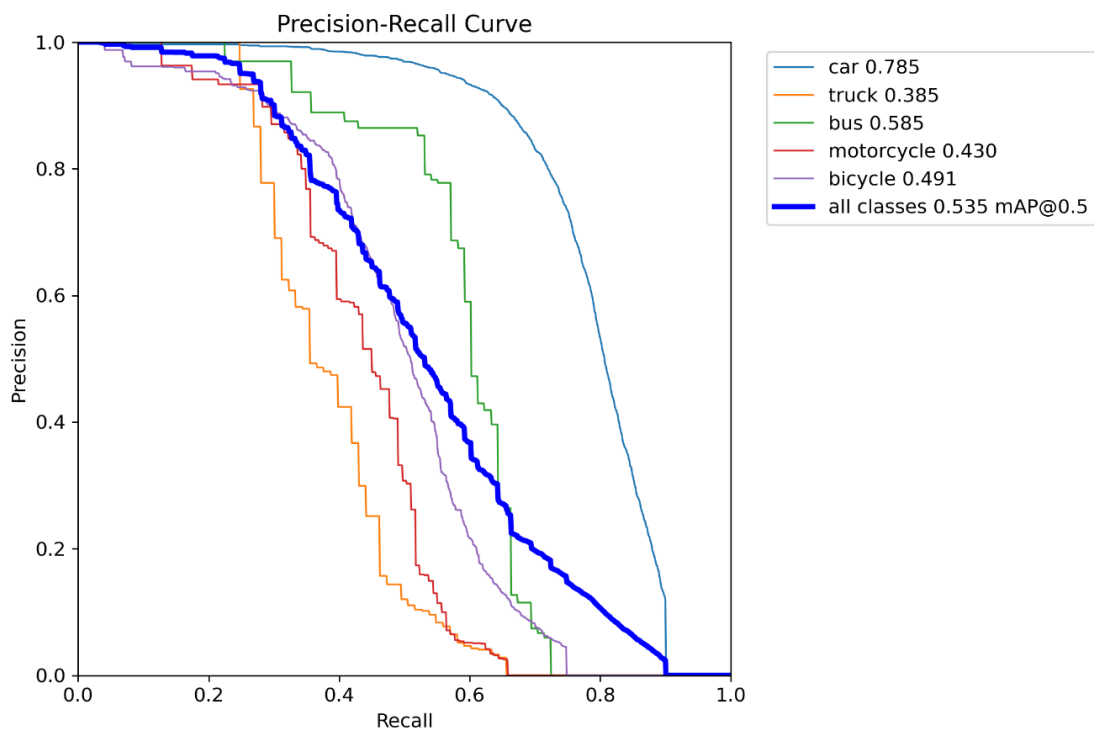


Figure 4.15 Precision-recall curve for YOLOv10s, $\beta = 0.01$ condition.

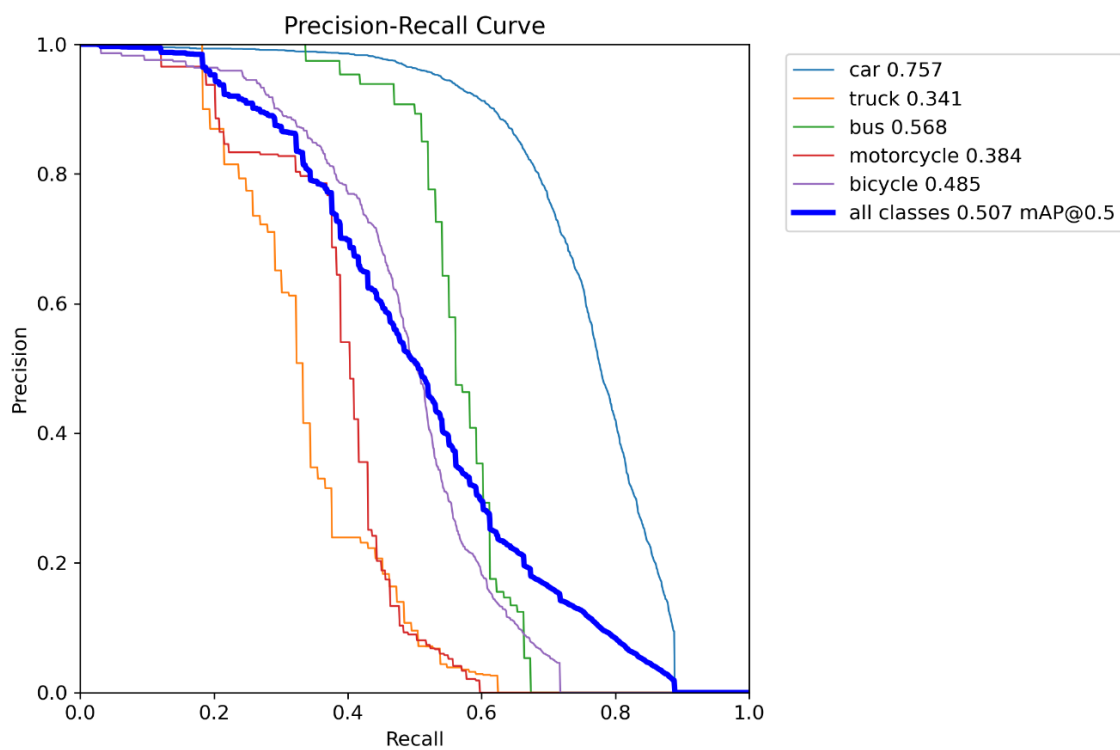


Figure 4.16 Precision-recall curve for YOLOv10s, $\beta = 0.02$ condition.

YOLOv10s (Figures 4.9–4.12) shows the same broad pattern as YOLOv8s — car with the strongest diagonal, other classes confused primarily with background — with one exception worth flagging rather than over-interpreting: the $\beta = 0.005$ confusion matrix shows a visibly different distribution from the other three conditions, with lower background-confusion values across every class and higher off-diagonal values between vehicle classes themselves (for example, car-truck and car-bus confusion, which are close to zero elsewhere). Since $\beta = 0.005$ is the lightest fog condition and its per-class instance counts are identical to the other stages (Table 3.1), this is more likely attributable to normal run-to-run variability at a single checkpoint — consistent with the single-training-run limitation noted in Section 5.4 — than to a systematic property of light fog specifically, but it is a visible enough difference that a repeated run at this stage would be a reasonable first step if this analysis is extended.

Across the full fog range now available (Clean to $\beta = 0.02$), truck and bus show the sharpest overall degradation (truck: -0.119 , bus: -0.116), while car is the most robust (-0.048). Motorcycle appears comparatively stable through $\beta = 0.01$ (-0.007) but declines further under the heaviest fog, dropping an additional 0.046 between $\beta = 0.01$ and $\beta = 0.02$ — suggesting its apparent resistance to light-to-moderate fog does not extend to the most severe condition. This broadly mirrors the pattern observed for YOLOv8s and again points to training-instance scarcity as a contributing factor independent of fog density.

4.1.3 Qualitative Detection Examples

Figure 4.17 shows representative ground-truth annotations for the Clean condition, which draws on a different source dataset from the foggy stages. Figure 4.18 shows the corresponding ground-truth annotations for the three foggy conditions ($\beta = 0.005$, $\beta = 0.01$, $\beta = 0.02$), which share a single reference scene varied only by synthetic fog density. Figures 4.19–4.22 and Figures 4.23–4.26 show each model’s predicted detections on comparable validation scenes for each of the four visibility conditions individually, illustrating typical detection behavior — correctly localized cars and bicycles alongside missed detections of partially occluded or distant objects, consistent with the recall values reported above and increasingly frequent as fog density increases.

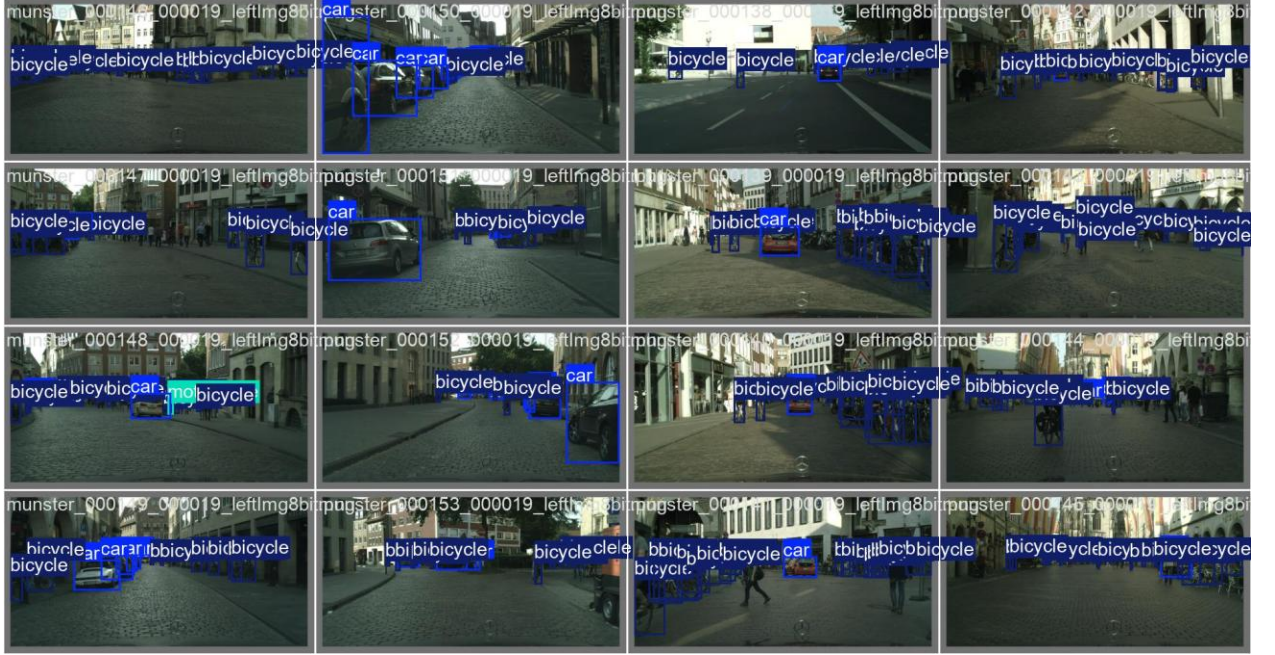


Figure 4.17 Ground-truth bounding boxes for a representative Clean-condition validation batch.



Figure 4.18 Ground-truth bounding boxes for the shared reference scene used across the three foggy conditions ($\beta = 0.005$, $\beta = 0.01$, $\beta = 0.02$).



Figure 4.19 YOLOv8s predicted detections, Clean condition.

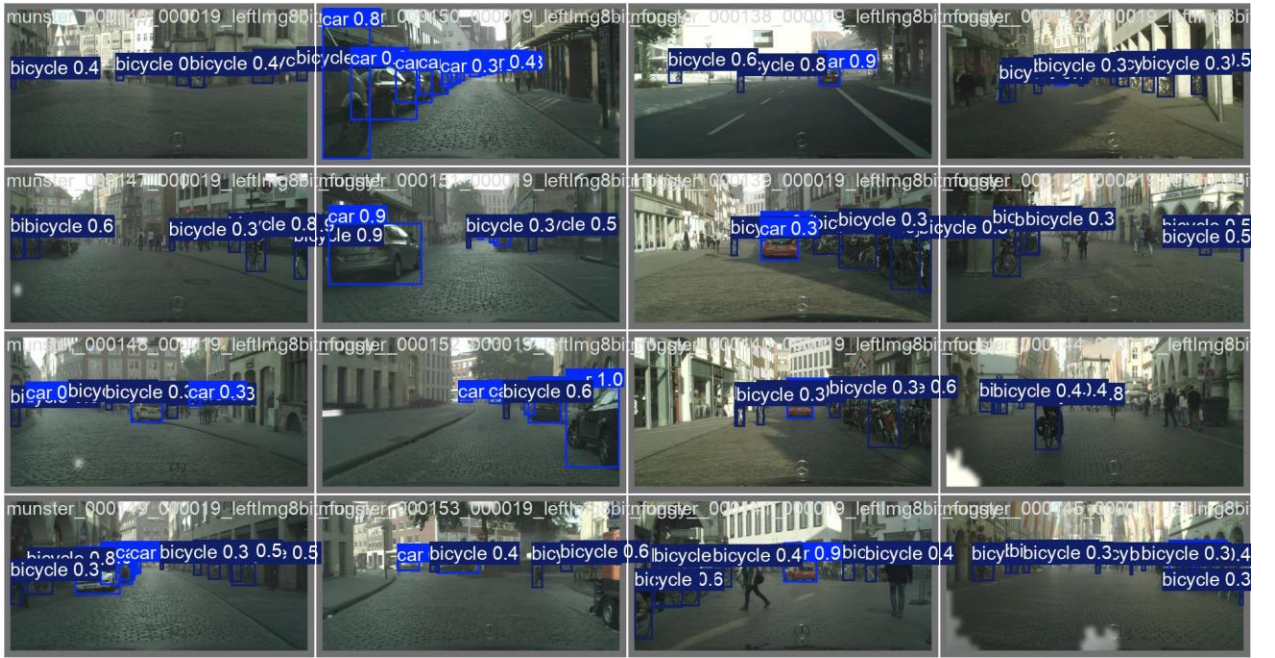


Figure 4.20 YOLOv8s predicted detections, $\beta = 0.005$ condition.



Figure 4.21 YOLOv8s predicted detections, $\beta = 0.01$ condition.



Figure 4.22 YOLOv8s predicted detections, $\beta = 0.02$ condition.



Figure 4.23 YOLOv10s predicted detections, Clean condition.



Figure 4.24 YOLOv10s predicted detections, $\beta = 0.005$ condition.



Figure 4.25 YOLOv10s predicted detections, $\beta = 0.01$ condition.



Figure 4.26 YOLOv10s predicted detections, $\beta = 0.02$ condition.

Visually, the images for both models show the expected qualitative progression across conditions: scene contrast and background detail degrade steadily from Clean through $\beta = 0.02$, and detected boxes become visibly sparser and lower-confidence in the denser fog images, particularly for objects toward the edges or rear of a scene. Bicycle is by far the

most frequently detected class across both sets of images, consistent with its high validation instance count relative to truck, bus, and motorcycle (Table 3.1); motorcycle detections are correspondingly rare across all eight images, appearing only occasionally and at lower confidence than car or bicycle detections in the same scene, a visual echo of the per-class mAP50 gap documented in Table 4.6. Consistent with Figures 4.9–4.12’s confusion-matrix pattern, YOLOv10s’s detections carry visibly higher confidence scores than YOLOv8s’s on comparable scenes across most conditions, though the gap narrows under the heaviest fog, mirroring the aggregate precision trend reported in Section 4.2.1.

4.1.4 Training Convergence Curves

Figures 4.27–4.30 and Figures 4.31–4.34 show the per-epoch training curves recorded by the Ultralytics training pipeline for each of the four curriculum stages individually, for YOLOv8s and YOLOv10s respectively: training and validation box, classification, and DFL losses, together with precision, recall, mAP50, and mAP50-95 on the validation set.

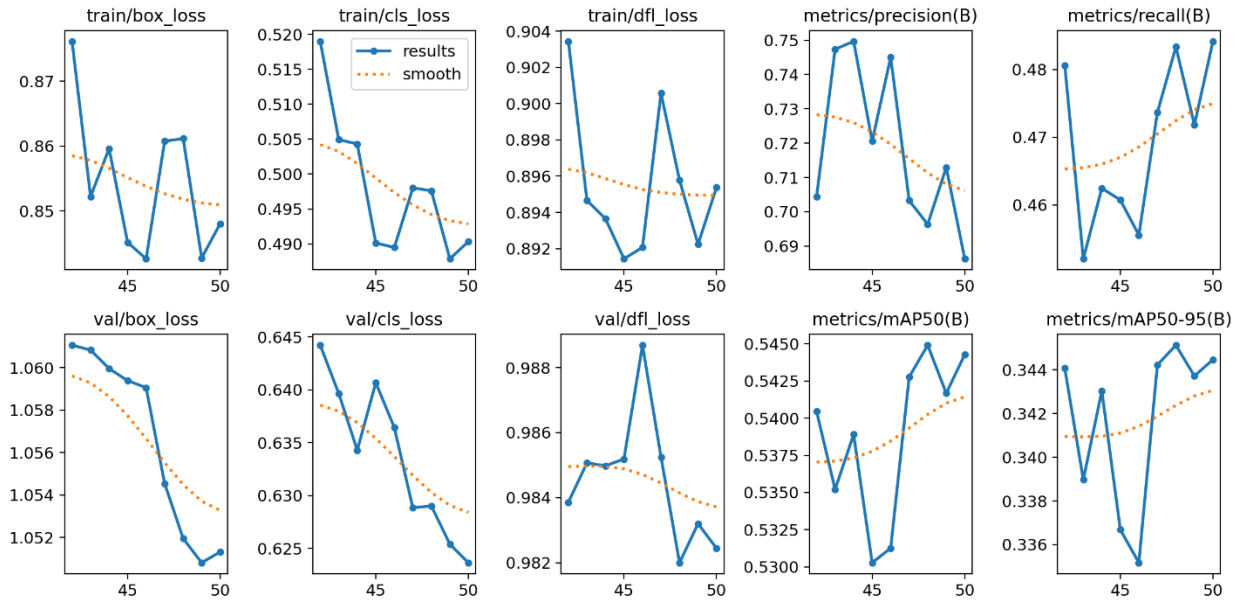


Figure 4.27 Training curves for YOLOv8s, Clean stage.

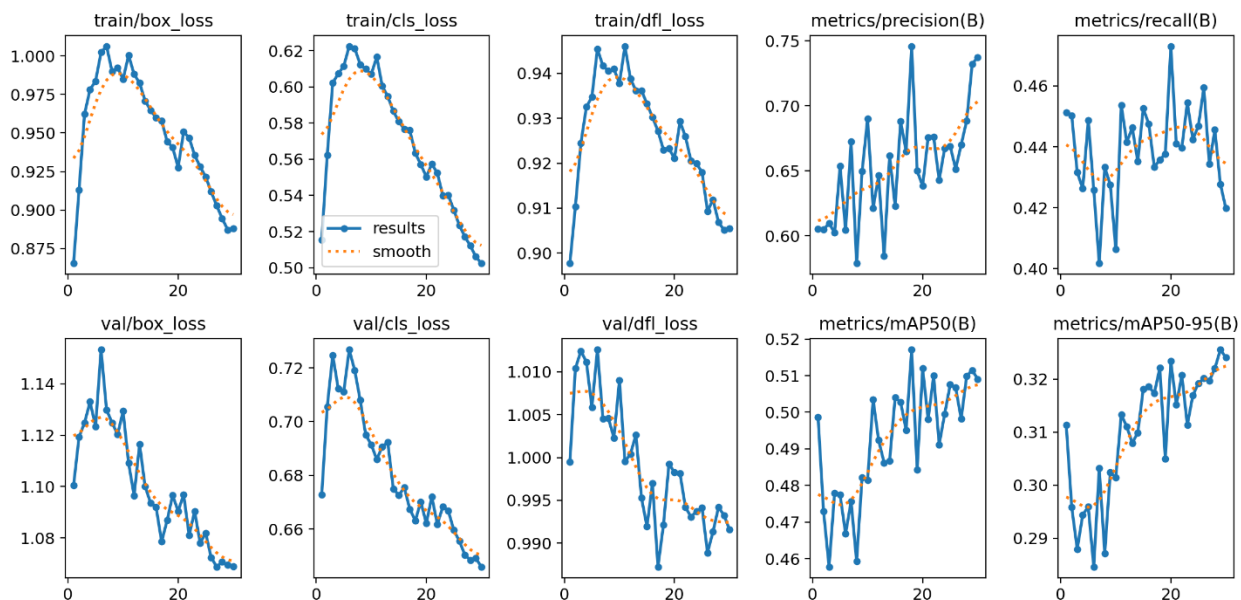


Figure 4.28 Training curves for YOLOv8s, $\beta = 0.005$ stage.

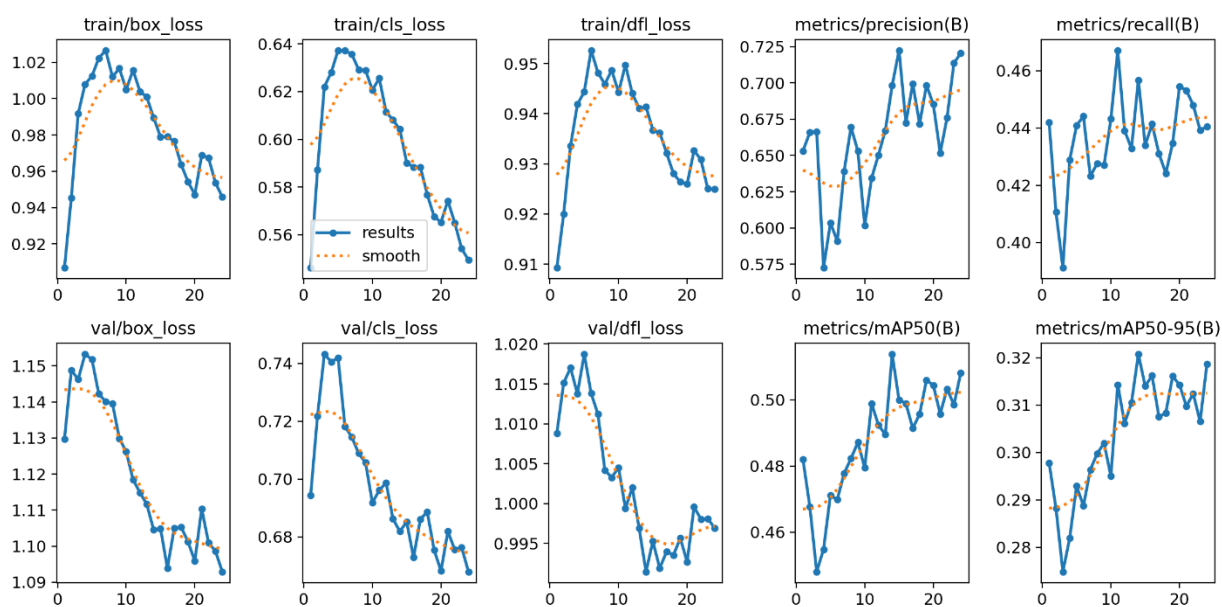


Figure 4.29 Training curves for YOLOv8s, $\beta = 0.01$ stage.

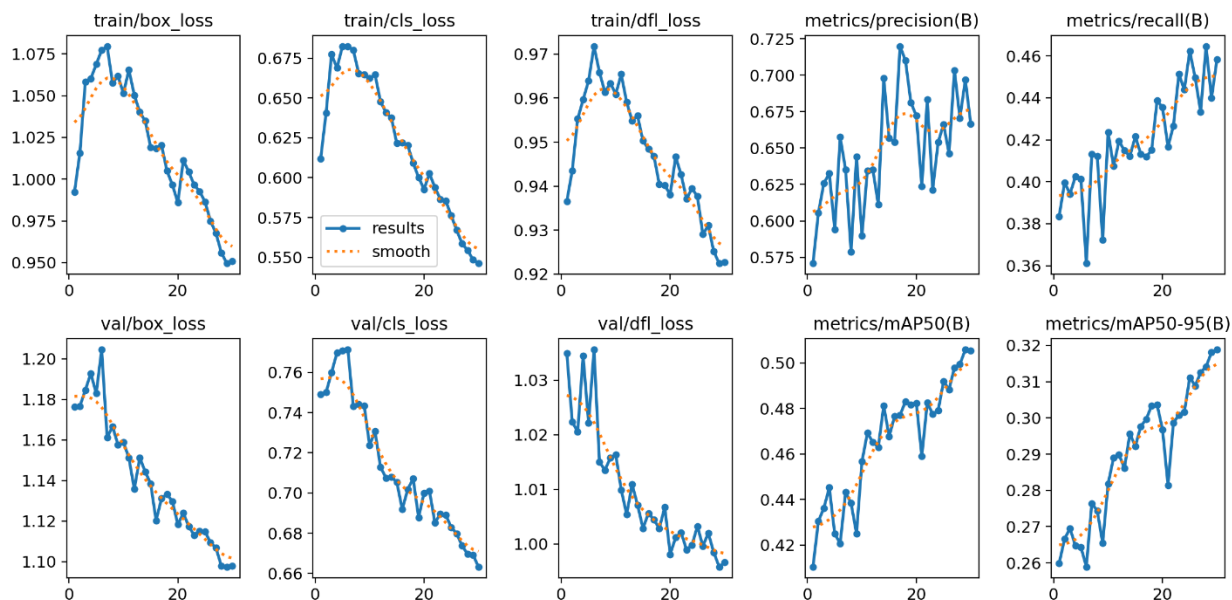


Figure 4.30 Training curves for YOLOv8s, $\beta = 0.02$ stage.

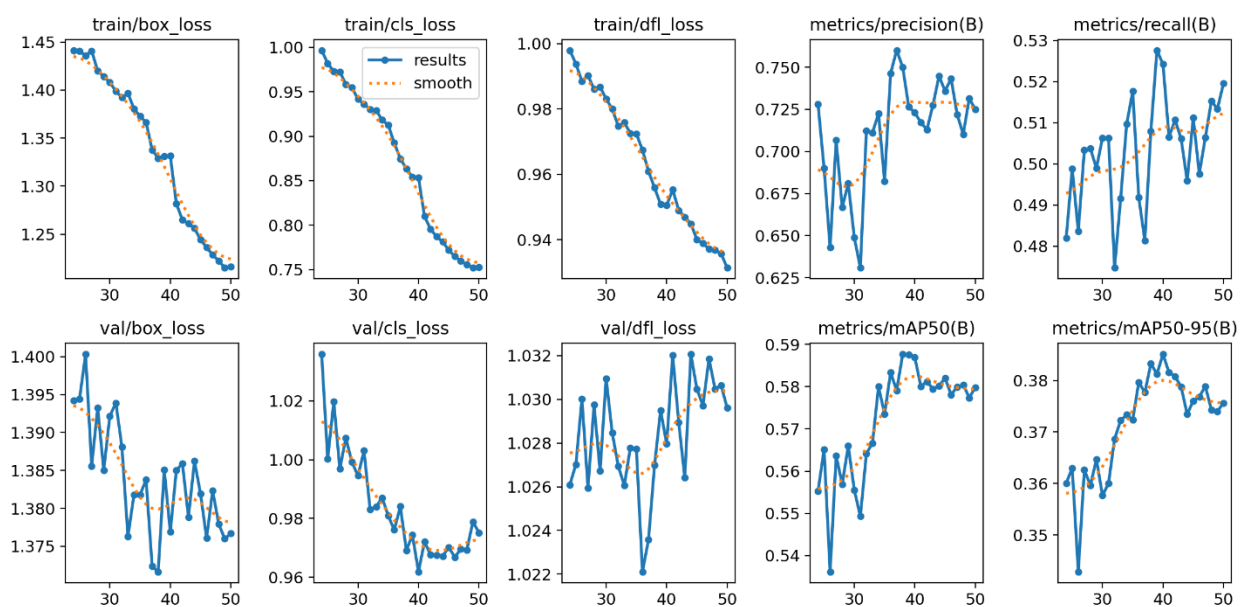


Figure 4.31 Training curves for YOLOv10s, Clean stage.

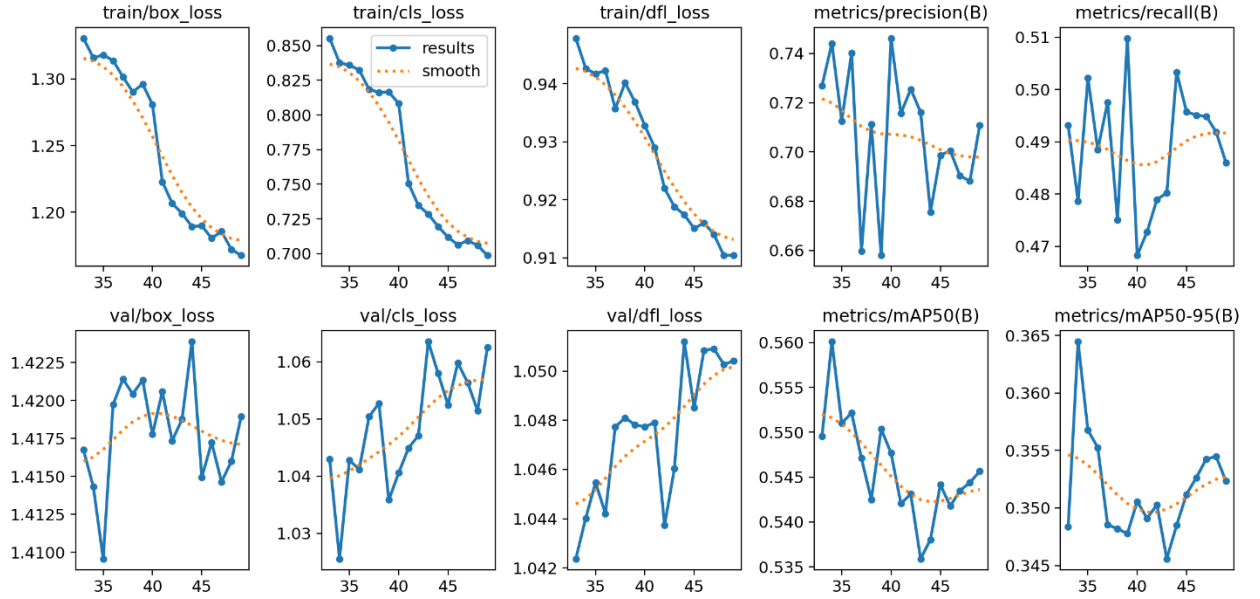


Figure 4.32 Training curves for YOLOv10s, $\beta = 0.005$ stage.

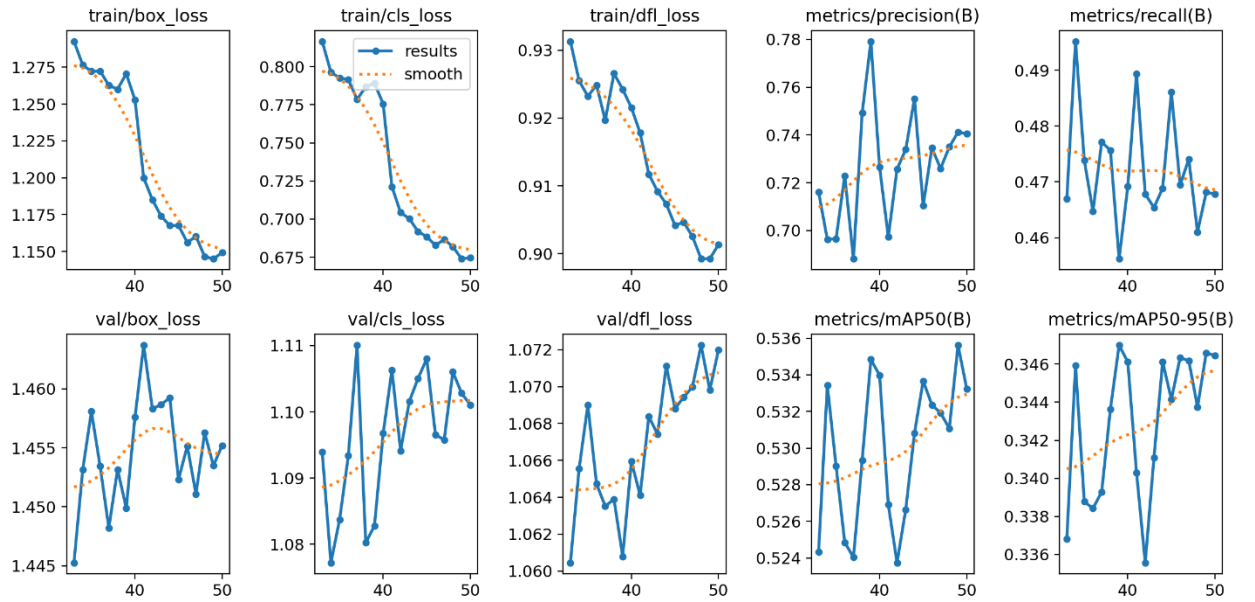


Figure 4.33 Training curves for YOLOv10s, $\beta = 0.01$ stage.

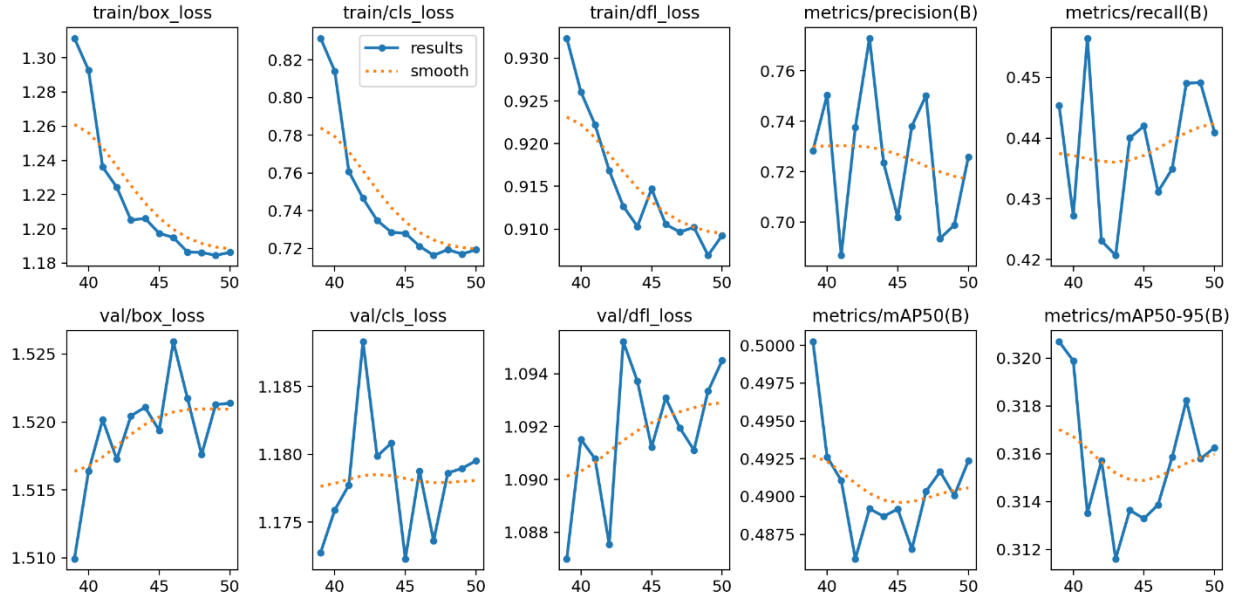


Figure 4.34 Training curves for YOLOv10s, $\beta = 0.02$ stage.

The images also reveal a difference in how each model’s curriculum was executed in practice. For YOLOv8s (Figures 4.27–4.30), the three foggy stages ($\beta = 0.005$, $\beta = 0.01$, $\beta = 0.02$) are each shown across a full, uninterrupted epoch range from initialization, consistent with Section 3.3.1’s description of each stage fine-tuning independently from the Stage 1 checkpoint; all three show the pattern expected of early fine-tuning from a pretrained checkpoint, with box, classification, and DFL losses rising briefly before declining as the model adapts to the shifted foggy input distribution. The Clean stage, by contrast, is shown only across its final epochs (roughly 42–50), a direct visual record of the Google Colab checkpoint-resume workflow described in Section 3.5: free-tier session limits periodically interrupted training, and the session resumed from the last saved checkpoint rather than restarting from scratch. For YOLOv10s (Figures 4.31–4.34), all four stages are shown as partial epoch ranges rather than complete runs — a pattern consistent with, and further visual confirmation of, the strictly sequential curriculum described in Section 3.3.2, in which each stage continues training from the previous stage’s checkpoint rather than restarting, compounded by the same session-resume interruptions affecting YOLOv8s.

In all four cases, validation mAP50 trends upward across the range shown and shows no sustained decline toward the final epochs, indicating that the fitness-weighted best-epoch selection described in Section 3.6.3 is selecting a checkpoint from a genuinely converged region of training rather than an early peak followed by overfitting. Precision and recall curves are visibly noisier than the loss curves across all four runs, an expected consequence of computing both metrics over a validation set containing very few instances of the rarest classes (Table 3.1); a single batch containing or omitting a handful of truck or bus instances can shift epoch-to-epoch precision and recall considerably more than it shifts an aggregate loss value averaged over all predictions. This observation is consistent with, and lends further support to, the class-imbalance-driven variability discussed throughout Section 4.4.

4.2 Comparative Analysis: YOLOv8s vs. YOLOv10s

Sections 4.1.1 and 4.1.2 reported YOLOv8s and YOLOv10s independently; this section brings the two sets of results together to characterize how the two models compare under the same evaluation protocol. Table 4.5 restates the overall mAP50 achieved by each model at each fog density, alongside the difference between them.

Table 4.5 Overall mAP50 comparison between YOLOv8s and YOLOv10s by stage.

Stage	YOLOv8s mAP50	YOLOv10s mAP50	Δ (v10s – v8s)
Clean	0.545	0.587	+0.042
$\beta = 0.005$	0.511	0.568	+0.057
$\beta = 0.01$	0.514	0.535	+0.021
$\beta = 0.02$	0.506	0.507	+0.001

YOLOv10s outperforms YOLOv8s at every stage, with the margin ranging from roughly 2 to 6 points of mAP50 under clean and light-to-moderate fog. The gap narrows sharply at the heaviest fog density ($\beta = 0.02$), where the two models converge to within 0.1 points of one another (50.6% vs. 50.7%). Combined with the pattern already noted in Sections 4.1.1 and 4.1.2 — YOLOv8s staying nearly flat across all three foggy conditions while YOLOv10s degrades monotonically with fog density — this convergence suggests that

under sufficiently dense fog, both models approach a similar practical ceiling on this dataset, and the advantage YOLOv10s shows elsewhere narrows to a level that no longer clearly favors either curriculum design.

YOLOv10s also achieves consistently higher precision than YOLOv8s at every stage (for example, 0.762 versus 0.667 at $\beta = 0.02$), while recall for both models stays comparatively similar and modest across all foggy conditions (0.44–0.52), indicating that missed detections under fog remain the dominant error mode for both architectures rather than false positives. It is worth stressing, however, that this is not a controlled architecture-only comparison: as described in Chapter 3, YOLOv10s was trained at a higher input resolution (832×832 versus 640×640), with a larger batch size, the AdamW optimizer, and the additional preprocessing pipeline of Section 3.2, none of which were applied to YOLOv8s. The results above should therefore be read as a comparison of two complete training pipelines rather than an isolated test of architecture alone; this point is revisited in Section 1.8.

4.2.1 Quantitative Comparison: Per-Class Detail

Table 4.5 compared the two models only at the aggregate, all-class level. Table 4.6 breaks this comparison down per class at the two extremes of the fog curriculum — Clean and the heaviest fog condition, $\beta = 0.02$ — using the per-class figures already reported in Tables 4.2 and 4.4.

Table 4.6 Per-class mAP50 comparison at Clean and $\beta = 0.02$, both models.

Class	v8s Clean	v10s Clean	v8s $\beta=0.02$	v10s $\beta=0.02$	Δ at $\beta=0.02$ (v10s–v8s)
Car	0.749	0.805	0.713	0.757	+0.044
Truck	0.406	0.460	0.378	0.341	−0.037
Bus	0.679	0.684	0.582	0.568	−0.014
Motorcycle	0.408	0.437	0.377	0.384	+0.007
Bicycle	0.482	0.548	0.477	0.485	+0.008

This breakdown reveals a pattern the aggregate mAP50 figures in Table 4.5 conceal. YOLOv10s leads YOLOv8s in every class under clean conditions, and continues to lead for car, motorcycle, and bicycle at the heaviest fog density. For truck and bus, however — the two least-represented classes in the training data (93 and 98 instances respectively; Table 3.1) — YOLOv8s actually overtakes YOLOv10s at $\beta = 0.02$, reversing the direction of the comparison entirely. This is consistent with, and adds further support to, the interpretation already developed in Section 4.4: under the combined stress of heavy fog and a very small training sample, whichever advantage YOLOv10s’s architecture and preprocessing pipeline provide elsewhere appears to break down specifically for the rarest classes, where there is simply too little training signal for either architecture to generalize reliably. The near-tie in aggregate mAP50 at $\beta = 0.02$ reported in Section 4.2 is therefore not a uniform tie across classes; it is an average that combines a YOLOv10s advantage on well-represented classes with a YOLOv8s advantage on the two most data-starved classes.

Precision and recall trends (Tables 4.1 and 4.3) reinforce this picture from a different angle. YOLOv10s’s precision advantage over YOLOv8s widens as fog increases (from roughly 3 points at Clean to over 9 points at $\beta = 0.02$), indicating that YOLOv10s becomes comparatively more conservative — producing fewer false-positive detections — as conditions worsen, whereas YOLOv8s’s precision degrades more sharply. Recall, in contrast, converges between the two models under heavy fog, suggesting that both architectures ultimately hit a similar ceiling on how many real objects they can find once contrast and edge information are sufficiently degraded, even though they reach that ceiling through different precision-recall trade-offs.

4.2.2 Computational Efficiency Comparison

At their respective published reference configurations (640×640 input resolution, the standard benchmark setting reported in the original architecture papers), YOLOv10s is the more computationally efficient design on paper: approximately 7.2 million parameters and 21.6 GFLOPs, compared to YOLOv8s’s 11.2 million parameters and 28.8 GFLOPs [5],

[6] — roughly 35% fewer parameters and 25% fewer FLOPs for YOLOv10s at matched resolution.

However, as noted in Section 3.1, YOLOv10s in this thesis was trained at a higher input resolution of 832×832 rather than the standard 640×640 , to improve detection of small and distant objects. Since computational cost in a convolutional network scales approximately with the square of the input resolution, this raises YOLOv10s’s effective compute in the configuration actually used in this study to approximately 36–37 GFLOPs ($21.6 \times (832/640)^2 \approx 36.5$ GFLOPs). This is an estimate obtained by scaling the published reference figure rather than a directly profiled measurement, and should be treated accordingly, but it indicates that the YOLOv10s configuration evaluated in this thesis likely required more computation per image than the YOLOv8s configuration, despite YOLOv10s having roughly 35% fewer parameters and being the more FLOP-efficient architecture at matched resolution. This is a further respect — beyond the differences in optimizer, batch size, and preprocessing already noted in Section 4.2 — in which the accuracy comparison in this chapter reflects two different complete training and inference configurations rather than a controlled, resolution-matched test of architectural efficiency. A fair efficiency comparison, left for future work, would require running both models at identical input resolution.

4.2.3 Qualitative Comparison

Figure 4.18 (Section 4.1.3), together with the $\beta = 0.01$ images from Figures 4.21 and 4.25, present ground truth, YOLOv8s predictions, and YOLOv10s predictions for the identical validation batch under the $\beta = 0.01$ fog condition, allowing direct visual comparison between the two models’ outputs on the same input images rather than on different scenes. Consistent with the quantitative pattern established in Section 4.2.1, YOLOv10s’s predictions (Figure 4.25) show visibly higher confidence scores on correctly localized vehicles and fewer spurious low-confidence boxes than YOLOv8s’s predictions (Figure 4.21) on the same batch, reflecting YOLOv10s’s higher measured precision at this fog density. Both models share a common failure pattern when compared against the ground truth (Figure 4.18): missed detections cluster around small, distant, or partially occluded vehicles, particularly bicycles and motorcycles near the edges of the frame or overlapping

with other objects, rather than being distributed uniformly across the scene. This is consistent with the recall values reported in Tables 4.1 and 4.3, which are lower than precision for both models at every fog stage, and supports the interpretation that missed detections — not false detections — are the dominant error mode for both architectures under fog, as discussed in Section 4.4.

Neither model produces confident false-positive detections on background regions such as fogged-out sky or road surface in the examples shown, which suggests that the anchor-free, decoupled detection heads shared by both architectures (Section 3.4.1) remain well-calibrated on background rejection even as fog degrades foreground object visibility; the practical cost of fog for both detectors is primarily paid in missed real objects rather than in spurious detections of non-objects.

4.3 Comparison with Published Benchmarks

To situate these results against prior work on the same benchmark, Table 4.7 compares the mAP50 achieved in this study against HR-YOLO [7], a YOLOv8-based detector proposed by Zhang and Jia that integrates a dedicated defogging network (DND-Net) and an attention-enhanced backbone (EHPD-Net), and that reports results on Foggy Cityscapes. HR-YOLO evaluates only at the heaviest fog severity (constant attenuation coefficient of 0.02), so the comparison is restricted to that condition. Two figures from HR-YOLO are relevant here: the final proposed model's mAP50, and the mAP50 of its own unmodified YOLOv8-s baseline, evaluated on the same benchmark before any of the proposed modules are added.

Table 4.7 mAP50 (%) at heavy fog ($\beta = 0.02$) against HR-YOLO's reported baseline and proposed model.

Model	mAP50 ($\beta = 0.02$)
This work — YOLOv8s (standard protocol)	50.6%
This work — YOLOv10s (preprocessing-enhanced)	50.7%
HR-YOLO's own YOLOv8-s baseline [7]	44.2%
HR-YOLO, proposed model with DND-Net + EHPD-Net [7]	49.5%

Two comparisons follow from this table. First, against HR-YOLO's fully proposed model, both models in this study exceed the reported 49.5% mAP50, by roughly 1.1 to 1.2 percentage points, despite neither YOLOv8s nor YOLOv10s here incorporating a dedicated defogging network or attention module. Second, and more directly comparable, is the baseline-to-baseline figure: HR-YOLO's own unmodified YOLOv8-s achieves 44.2% mAP50 on the same benchmark at the same fog density, roughly 6.4 points below the 50.6% obtained by the YOLOv8s model trained in this study under the standard four-stage curriculum. Since both figures come from the same base architecture on the same dataset and fog condition, this is the closer of the two comparisons, and the gap is plausibly attributable at least in part to the curriculum fine-tuning strategy used here, rather than to any architectural difference, since none exists between the two YOLOv8-s baselines being compared.

This comparison should still be read with caution rather than as a direct head-to-head result. HR-YOLO evaluates across the full eight-class Cityscapes taxonomy (person, rider, car, truck, bus, train, motorbike, bicycle), which includes two non-vehicle classes and one class (train) not present in this study's five-class target set, so the two mAP50 figures are not computed over an identical label space. HR-YOLO also does not report the specific training duration, image resolution, or optimizer settings used for its YOLOv8-s baseline on Foggy Cityscapes specifically, whereas its RTTS experiments used 300 epochs and the Adam optimizer at 640×640 resolution; if a similarly extended schedule was used for the Foggy Cityscapes baseline, the higher mAP50 obtained in this study cannot be attributed

to curriculum fine-tuning alone. The comparison is offered as a useful reference point rather than a controlled result.

A separate body of related work reports substantially lower mAP50, in the 35–47% range, on Foggy Cityscapes under unsupervised domain adaptation, where the detector never sees labeled foggy images during training [32], [33], [34]. Because that is a materially harder learning setting, those figures are not included in Table 4.6; the gap between this study's results and domain-adaptation baselines should be attributed primarily to the difference in supervision, not treated as evidence that curriculum fine-tuning is inherently superior to domain adaptation as a method.

4.4 Discussion

Two patterns recur across both models and are worth drawing out explicitly. First, per-class performance is ordered the same way in both models — car performs best, followed by bus, bicycle, motorcycle, and truck — and this ordering matches the number of training instances available per class far more closely than it matches fog density (Section 4.1.1). This indicates that, on this dataset, class imbalance is at least as significant a limiting factor as fog itself, and that further gains for the weakest classes (truck, motorcycle) are more likely to come from addressing data scarcity than from further fog-specific tuning. Notably, the same ordering (car > bus > bicycle > motorcycle > truck) also holds for YOLOv10s at the heaviest fog density, $\beta = 0.02$ (Table 4.4), which is the most demanding visibility condition tested in this study; the fact that class imbalance still dominates the per-class ranking even at maximum fog severity is further evidence that it is the primary bottleneck rather than a secondary effect that fog might be expected to override.

Second, the two models respond differently to the curriculum's staged fog ordering. YOLOv10s, trained sequentially with each stage initialized from the previous one (Section 3.3.2), shows a clean monotonic decline in mAP50 as fog density increases, consistent with the curriculum's intended difficulty progression. YOLOv8s, with each foggy stage fine-tuned independently from the same clean checkpoint (Section 3.3.1), shows almost no difference between light and moderate fog. This suggests that the sequential fine-tuning strategy, and not the architecture alone, is a meaningful contributor to how closely a

model's performance tracks fog severity — an interaction between training strategy and outcome that is easy to overlook when comparing only final mAP50 figures.

Precision remained higher than recall for both models across nearly every stage, indicating that the dominant failure mode under fog is missed detections rather than spurious ones. This is consistent with the physical effect of fog described in Chapter 3: reduced contrast and edge definition make objects harder to detect at all, rather than making the model more prone to reporting objects that are not present. It is also worth noting, for context, that HR-YOLO reports a considerably higher mAP of 79.8% on RTTS [40], a real-world foggy-scene dataset, underscoring that reported fog-detection accuracy depends heavily on which benchmark is used and is not directly transferable between real and synthetic fog datasets.

4.4.1 Practical Implications

Beyond the architectural comparison itself, three findings in this chapter have direct bearing on deploying a system of this kind in practice, for example as a fog-aware traffic monitoring aid on roads in Bangladesh, where seasonal fog is common during winter months. First, the class-imbalance finding (Section 4.4) suggests that the most cost-effective way to improve a deployed detector's reliability for rare-but-safety-critical categories such as motorcycles and bicycles is likely to be collecting or oversampling additional rare-class training data specific to the deployment region, rather than investing further in fog-specific architectural modification or more aggressive preprocessing. Second, the computational efficiency analysis (Section 4.2.2) indicates that if YOLOv10s is deployed at the higher 832×832 resolution used in this study, its practical inference cost is not automatically lower than YOLOv8s's despite YOLOv10s's smaller parameter count; a deployment targeting resource-constrained hardware, such as a roadside edge device, would need to weigh the accuracy gained from the higher resolution against this increased per-frame compute cost, or run YOLOv10s at the standard 640×640 resolution to realize its full efficiency advantage. Third, because precision remains higher than recall for both models across all fog conditions (Section 4.2.1), a deployed system is more likely to silently miss a vehicle under fog than to falsely flag one; for a safety-critical application this asymmetry may warrant a lower confidence threshold at inference time than would be

used under clear conditions, accepting more false positives in exchange for fewer missed detections, particularly for the rare vehicle classes shown in this chapter to be most at risk.

This chapter reported the quantitative and qualitative results for YOLOv8s and YOLOv10s across all four curriculum stages, compared the two architectures in detail, situated the findings against published benchmarks, and discussed their implications. Chapter 5 now concludes the thesis, summarizing the study, reviewing the research objectives, and outlining its limitations and future work.

CHAPTER 5

CONCLUSION AND FUTURE WORK

This chapter closes the thesis: it summarizes the study, reviews how each research objective from Section 1.4.2 was addressed, restates the major findings, revisits the study’s limitations, offers recommendations, outlines future work, and ends with final remarks.

5.1 Summary of the Study

This thesis presented a multi-stage curriculum training study of vehicle detection under synthetic fog, comparing YOLOv8s trained under a standard protocol against YOLOv10s trained with an additional preprocessing pipeline, across four visibility conditions (Clean, $\beta = 0.005$, $\beta = 0.01$, $\beta = 0.02$) and five vehicle classes on the Cityscapes and Foggy Cityscapes benchmarks. Motivated by the broader goal of more weather-robust camera-based perception for autonomous vehicles (Section 1.3), the study kept both detector architectures unmodified and isolated the effects of curriculum training and lightweight classical preprocessing, rather than following the architecturally complex, dehazing-network-heavy approach common in prior fog-detection work (Chapter 2).

5.2 Achievement of Research Objectives

Objective 1 (annotation and curriculum pipeline): Achieved. A reproducible pipeline was built that converts Cityscapes polygon annotations to YOLO format and organizes Clean and three Foggy Cityscapes severities into a shared four-stage curriculum (Sections 3.1–3.3).

Objective 2 (training both models): Achieved. YOLOv8s was trained under the standard four-stage protocol and YOLOv10s under the same curriculum with an added Gamma correction, CLAHE, and rare-class oversampling pipeline, with neither architecture modified (Sections 3.2, 3.4).

Objective 3 (per-condition, per-class evaluation): Achieved. Precision, Recall, mAP50, and mAP50-95 were reported for both models at every curriculum stage, with per-class breakdowns for all four conditions (Sections 4.1–4.2).

Objective 4 (comparison against benchmarks): Achieved. The two architectures were compared quantitatively, qualitatively, and computationally (Section 4.2), and situated against HR-YOLO, the closest comparable published benchmark on Foggy Cityscapes (Section 4.3).

Objective 5 (class imbalance analysis): Achieved. The per-class analysis in Section 4.2.1 and the Discussion in Section 4.4 identify training-instance scarcity, rather than fog density, as the dominant factor separating strong- and weak-performing classes.

5.3 Major Findings

The central findings are fourfold. First, YOLOv10s outperformed YOLOv8s in aggregate mAP50 at every fog density tested, though the two converge closely under the heaviest fog; per-class analysis (Section 4.2.1) shows this aggregate convergence conceals a more specific pattern, with YOLOv8s actually exceeding YOLOv10s on the two most data-scarce classes, truck and bus, at $\beta = 0.02$, even as YOLOv10s retains its advantage on car, motorcycle, and bicycle. As with the overall comparison, this reflects differences in the full training pipeline rather than architecture alone. Second, at matched fog severity and evaluation protocol, the YOLOv10s pipeline exceeded a comparable supervised benchmark from the published literature, HR-YOLO [7], at the heaviest fog density. Third, class imbalance rather than fog density appears to be the dominant factor separating strong- and weak-performing vehicle classes in both models, with rare classes such as truck and motorcycle trailing car and bus consistently across all visibility conditions, a pattern that holds even at the heaviest fog density tested. Fourth, YOLOv10s’s favorable efficiency profile relative to YOLOv8s does not necessarily carry over to the configuration used in this study: because YOLOv10s was trained at a higher input resolution than its published reference configuration, its estimated computational cost in this thesis’s setup is likely higher, not lower, than YOLOv8s’s, despite YOLOv10s having roughly 35% fewer parameters (Section 4.2.2).

5.4 Limitations

The following methodological limitations apply to this study as a whole and should be kept in mind when interpreting the findings above.

Non-matched training conditions. YOLOv8s and YOLOv10s were not trained under identical conditions (Section 4.2). Differences in input resolution, batch size, optimizer, and the presence or absence of the preprocessing pipeline (Section 3.2) mean the comparison in this thesis reflects two complete pipelines rather than architecture in isolation. A controlled ablation — training YOLOv10s without preprocessing at 640×640 , or YOLOv8s with the preprocessing pipeline at 832×832 — would be needed to isolate the contribution of architecture from that of preprocessing.

No cross-condition evaluation matrix. Each trained model was evaluated only on the validation set matching its own training condition (Section 3.6.2). The full 4×4 cross-condition matrix originally proposed — testing every model against every fog density, including out-of-distribution conditions — was not completed, so the domain gap for clean-trained models under fog, and the generalization of fog-trained models to other fog densities, cannot be directly quantified from this study.

Single training run per stage. Each stage was trained once, without repeated runs or reported variance. The mAP50 differences discussed in Section 4.2, particularly the narrow gap between models at the heaviest fog density, should therefore be treated as point estimates rather than statistically validated differences.

Synthetic fog only. All fog conditions in this study are synthetically rendered (Section 3.1); performance under real-world fog, which introduces additional effects such as non-uniform density and sensor noise not modeled by the Foggy Cityscapes rendering pipeline, was not evaluated.

Fog only, and benchmark data only. As discussed in Section 1.3, this study addresses fog exclusively and does not evaluate rain, snow, or low-light conditions, nor does it test on live vehicle-camera footage; both are necessary next steps toward the broader motivating goal of weather-robust autonomous-vehicle perception.

5.5 Recommendations

Based on the findings above, three recommendations follow for practitioners and researchers working on similar fog-robust detection problems. First, when training data for a target deployment region is limited for certain vehicle classes, prioritizing rare-class data

collection or oversampling is likely to yield a larger improvement than further fog-specific architectural or preprocessing changes, since class imbalance was found to be at least as significant a limiting factor as fog itself (Section 4.4). Second, practitioners comparing detector architectures for a fog-robustness use case should ensure comparisons are made at matched input resolution, since this study found that YOLOv10s’s published efficiency advantage did not hold at the higher resolution used here (Section 4.2.2); resolution should be treated as an explicit, controlled variable rather than left to differ incidentally between models. Third, because precision consistently exceeded recall for both models under fog (Section 4.2.1), a deployed system prioritizing safety should consider a lower confidence threshold under detected or expected fog conditions, accepting more false positives in exchange for fewer missed detections, particularly for rare vehicle classes.

5.6 Future Work

Future work could address the limitations identified in Section 5.4 directly: a matched-condition ablation isolating architecture from preprocessing and resolution; completion of the full 4×4 cross-condition evaluation matrix originally proposed; a resolution-matched efficiency comparison to isolate architectural FLOP-efficiency from the effect of input resolution; repeated training runs to establish variance around the reported metrics; and evaluation on real-world foggy imagery to test whether the gains observed under synthetic fog transfer to genuine adverse-weather conditions. Beyond addressing these limitations, and in line with the broader motivation for this research (Section 1.3), the most direct extension of this work is to the other adverse-weather conditions outside its current scope: rain, snow, and low-light or nighttime conditions, each of which degrades detection through different mechanisms (Section 2.7) and would likely require its own curriculum design and preprocessing choices rather than a direct reuse of the fog-specific pipeline developed here. A longer-term extension would move beyond benchmark datasets entirely, toward real vehicle-camera footage and eventually integration within a fuller perception stack, as a step toward the camera-based, weather-robust autonomous vehicle perception that motivates this line of research.

5.7 Final Remarks

Fog remains a solvable but unsolved problem for camera-based vehicle detection, and this thesis has aimed to show that meaningful, measurable progress on that problem does not necessarily require adding architectural complexity: a carefully designed training curriculum and lightweight preprocessing, applied to unmodified, publicly available detectors, achieved fog-robustness results competitive with more elaborate published approaches. Class imbalance emerged as at least as important a factor as fog density itself, a finding that redirects attention toward data curation as much as toward model or algorithm design. There is still considerable distance between this study’s benchmark-dataset results and a deployable, weather-robust autonomous-vehicle camera system, but the reproducible pipeline, findings, and open questions identified here are offered as one incremental step toward that longer-term goal.

References

- [1] C. Ma and F. Xue, "A Review of Vehicle Detection Methods Based on Computer Vision," *Journal of Intelligent and Connected Vehicles*, vol. 7, pp. 1–18, 2024.
- [2] S. Grigorescu, B. Trasnea, T. Cocias, and G. Macesanu, "A Survey of Deep Learning Techniques for Autonomous Driving," *Journal of Field Robotics*, vol. 37, no. 3, pp. 362–386, 2020.
- [3] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, "The Cityscapes Dataset for Semantic Urban Scene Understanding," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 3213–3223.
- [4] C. Sakaridis, D. Dai, and L. Van Gool, "Semantic Foggy Scene Understanding with Synthetic Data," *International Journal of Computer Vision*, vol. 126, no. 9, pp. 973–992, 2018.
- [5] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," Ultralytics, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [6] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-Time End-to-End Object Detection," in *Proc. 38th Conf. Neural Information Processing Systems (NeurIPS)*, 2024.
- [7] Y. Zhang and N. Jia, "A Target Detection Model HR-YOLO for Advanced Driver Assistance Systems in Foggy Conditions," *Scientific Reports*, vol. 15, p. 13067, 2025.
- [8] D. Neumeister and D. Pape, "Automated Vehicles and Adverse Weather: Final Report," Report No. FHWA-JPO-19-755, U.S. Department of Transportation, Federal Highway Administration, Intelligent Transportation Systems Joint Program Office, 2019.

- [9] F. Sezgin, D. Vriesman, D. Steinhauser, R. Lugner, and T. Brandmeier, "Safe Autonomous Driving in Adverse Weather: Sensor Evaluation and Performance Monitoring," arXiv:2305.01336, 2023.
- [10] M. Hnewa and H. Radha, "Object Detection Under Rainy Conditions for Autonomous Vehicles: A Review of State-of-the-Art and Emerging Techniques," IEEE Signal Processing Magazine, vol. 38, no. 1, pp. 53–67, 2021.
- [11] G. Rjoub, O. A. Wahab, J. Bentahar, and A. S. Bataineh, "Improving Autonomous Vehicles Safety in Snow Weather Using Federated YOLO CNN Learning," in Mobile Web and Intelligent Information Systems (MobiWIS 2021), Lecture Notes in Computer Science, vol. 12814, Springer, 2021, pp. 121–134.
- [12] C. Guo et al., "Zero-Reference Deep Curve Estimation for Low-Light Image Enhancement," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2020, pp. 1780–1789.
- [13] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2014, pp. 580–587.
- [14] R. Girshick, "Fast R-CNN," in Proc. IEEE Int. Conf. Computer Vision (ICCV), 2015.
- [15] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 39, pp. 1137–1149, 2017.
- [16] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in Proc. IEEE Int. Conf. Computer Vision (ICCV), 2017, pp. 2980–2988.
- [17] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-Sampling Technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

- [18] S. S. A. Zaidi et al., "A Survey of Modern Deep Learning Based Object Detection Models," *Digital Signal Processing*, vol. 126, p. 103514, 2022.
- [19] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [20] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 7263–7271.
- [21] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," *arXiv:1804.02767*, 2018.
- [22] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," *arXiv:2004.10934*, 2020.
- [23] K. He, J. Sun, and X. Tang, "Single Image Haze Removal Using Dark Channel Prior," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 12, pp. 2341–2353, 2010.
- [24] J.-P. Tarel and N. Hautière, "Fast Visibility Restoration from a Single Color or Gray Level Image," in *Proc. IEEE 12th Int. Conf. Computer Vision (ICCV)*, 2009, pp. 2201–2208.
- [25] B. Li, X. Peng, Z. Wang, J. Xu, and D. Feng, "AOD-Net: All-in-One Dehazing Network," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 4770–4778.
- [26] K. Zuiderveld, "Contrast Limited Adaptive Histogram Equalization," in *Graphics Gems IV*, P. S. Heckbert, Ed. Academic Press, 1994, pp. 474–485.
- [27] E. Bıyık et al., "From Fog to Failure: The Unintended Consequences of Dehazing on Object Detection in Clear Images," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

- [28] W. Liu et al., "Detection-Friendly Dehazing: Object Detection in Real-World Hazy Scenes," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 3, pp. 3023–3036, 2023.
- [29] L. Cheng, D. Zhang, and Y. Zheng, "Road Object Detection in Foggy Complex Scenes Based on Improved YOLOv8," *IEEE Access*, vol. 12, pp. 107420–107430, 2024.
- [30] X. Meng, Y. Liu, L. Fan, and J. Fan, "YOLOv5s-Fog: An Improved Model Based on YOLOv5s for Object Detection in Foggy Weather Scenarios," *Sensors*, vol. 23, no. 11, p. 5321, 2023.
- [31] L. Wang, H. Qin, X. Zhou, X. Lu, and F. Zhang, "R-YOLO: A Robust Object Detector in Adverse Weather," *IEEE Transactions on Instrumentation and Measurement*, vol. 72, pp. 1–11, 2022.
- [32] Y. Chen et al., "Domain Adaptation Based Object Detection for Autonomous Driving," *arXiv:2307.09676*, 2023.
- [33] X. Zhao et al., "DAGL-Faster: Domain Adaptive Faster R-CNN for Vehicle Detection in Rainy and Foggy Weather," *Expert Systems with Applications*, 2023.
- [34] H. Wang et al., "SS-WDA: Unsupervised Domain Adaptive 3-D Object Detection from Clear to Fog," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2024.
- [35] G. Li, Z. Ji, X. Qu, R. Zhou, and D. Cao, "Cross-Domain Object Detection for Autonomous Driving: A Stepwise Domain Adaptive YOLO Approach," *IEEE Transactions on Intelligent Vehicles*, vol. 7, pp. 603–615, 2022.
- [36] X. Li et al., "Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection," in *Proc. Conf. Neural Information Processing Systems (NeurIPS)*, 2020.
- [37] Z. Zheng, P. Wang, D. Ren, W. Liu, R. Ye, Q. Hu, and W. Zuo, "Enhancing Geometric Factors in Model Learning and Inference for Object Detection and Instance Segmentation," *IEEE Transactions on Cybernetics*, vol. 52, pp. 8574–8586, 2021.

- [38] A. Vaswani et al., "Attention Is All You Need," in Proc. Conf. Neural Information Processing Systems (NeurIPS), 2017.
- [39] C. Feng et al., "TOOD: Task-Aligned One-Stage Object Detection," in Proc. IEEE Int. Conf. Computer Vision (ICCV), 2021.
- [40] B. Li, W. Ren, D. Fu, D. Tao, D. Feng, W. Zeng, and Z. Wang, "Benchmarking Single-Image Dehazing and Beyond," IEEE Transactions on Image Processing, vol. 28, no. 1, pp. 492–505, 2019.

Appendix